

A review on declarative approaches for constrained clustering

Thi-Bich-Hanh Dao^{*}, Christel Vrain^{*}

Univ. Orléans, INSA Centre Val de Loire, LIFO EA 4022, France

ARTICLE INFO

Keywords:

Clustering: distance-based clustering,
conceptual clustering
Constrained clustering
Integer Linear Programming (ILP)
Constraint Programming (CP)
Boolean satisfiability (SAT)

ABSTRACT

Clustering is an important Machine Learning task, which aims at discovering the implicit structure of data. Applying a clustering algorithm is easy but since clustering is an unsupervised task, tuning it so that the results is appropriate to the expert expectations is much less obvious. To overcome this, expert knowledge can be integrated into a clustering process; this is generally formalized as constraints on the desired output, thus leading to constrained clustering. There are two lines of research for clustering: distance based clustering, where data are grouped into clusters according to their dissimilarity and conceptual clustering, where a cluster must be a concept that is a set of objects and a set of properties that describe them. This second approach relies on Formal Concept Analysis and benefits from advances in Pattern Mining. [66] has shown the interest of declarative approaches for pattern mining and has led to a new research direction for clustering that is interested in the use of declarative frameworks, such as Integer Linear Programming, Constraint Programming or SAT. This has several advantages: finding a global optimum, integrating different kinds of constraints, even complex ones in a clustering process and even combining conceptual and distance-based clustering. In this paper we present an inventory of constraints and a survey of declarative methods for constrained clustering.

1. Introduction

Clustering is a well-known task in Machine Learning that aims at grouping data into clusters according to their similarity. It has been long studied and many algorithms have been developed, among them the well-known K-means algorithm [54,55] that aims at finding a partition of the objects into K clusters minimizing the within-cluster sum of squares criterion (WCSS) also called the sum of squared errors (SSE).

A clustering problem is often formulated as an optimization problem as in K-Means and different optimization criteria have been proposed, which aims at finding compact clusters (minimizing WCSS or minimizing the largest diameter of the clusters) or well-separated clusters (maximizing the split between clusters). Most optimization problems are NP-Hard and classic methods rely on heuristic and search for a local optimum. Density-based methods, as for instance DBscan [36] do not rely on an optimization criterion: a cluster is defined as a dense region and the approaches usually compute the neighborhood of each instance to determine whether it contains a sufficient number of points to be considered as dense.

These methods are called distance-based clustering: the distance between objects is a key ingredient, which, when combined to the optimization or the density criterion determines the results of the process. Applying a clustering algorithm is easy but since clustering is an unsupervised task, tuning it so that the results is appropriate to the expert expectations is much less obvious: first,

^{*} Corresponding authors.

E-mail addresses: thi-bich-hanh.dao@univ-orleans.fr (T.-B.-H. Dao), christel.vrain@univ-orleans.fr (C. Vrain).

<https://doi.org/10.1016/j.ijar.2024.109135>

Received 21 March 2023; Received in revised form 15 July 2023; Accepted 30 January 2024

Available online 2 February 2024

0888-613X/© 2024 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

it requires for the experts to decide which features to use for the computation of the distance, to choose the “right” optimization criterion, and then, to interpret the output of the algorithm, which is difficult, since the result is only groups of data instances put together based on their pairwise distances.

On the other hand there exists another subdomain of clustering, called conceptual clustering [59,37] that aims at finding concepts, where a concept is a set of data instances (a cluster) with a set of properties describing them, making the interpretation of the clusters easier. The criteria for assessing the quality of a cluster are based on the capacity to find properties of the clusters that are characteristic of instances belonging to the cluster and discriminant w.r.t. objects belonging to other clusters. Such methods are mostly designed for binary or categorical data.

Clustering is seen as an exploratory step in Machine Learning, helping at understanding the structure of the data. Nevertheless, often experts have some knowledge on their data, expecting for instance some points to be clustered together or apart and the result does not always fit their expectations. In this case, the experts must restart the whole clustering process, while changing the parameters, the features, or the optimization criterion, in order to find a clustering fitting their expectations. This task is complex, all the more that the output of a clustering process is difficult to interpret in case of distance-based methods. This has led to a new domain, called Constrained Clustering, that aims at introducing expert knowledge in the clustering process. Such knowledge is expressed by means of constraints; the simplest and the most studied constraints are must-link (resp. cannot-link) constraints between pairs of points stating that these points must be (resp. cannot be) in the same cluster. The exploratory nature of clustering and the integration of constraints suggest a *human-in-the-loop* process where an initial partition is built and depending on the results, the experts propose new constraints. Then either a new partition is built from scratch taking into account the new constraints, or the current partition is modified according to these new constraints.

In 2008, [66] showed that declarative frameworks (Constraint Programming in their work) could be an interesting paradigm for itemset mining and their work has then been extended to k-pattern-set mining [43,56] with an application to conceptual clustering. Later on [34] proposed a SAT encoding for a 2way-clustering. Declarative frameworks have many advantages compared to other frameworks. First they allow to integrate complex constraints, far beyond the classic must-link and cannot-link constraints. Second in such frameworks the satisfaction of all the constraints can be ensured. Moreover it is easy to integrate different views of the data, and even to integrate in a single framework conceptual and distance based clustering. However declarative frameworks suffer from computational complexity.

In this paper we present an inventory of constraints and a survey of declarative methods in Constrained Clustering. The constraints have been collected from different branches of constrained clustering (distance based, conceptual and descriptive clustering); they are organized into several categories: structural, semantic and conceptual constraints. Such constraints allow to express the knowledge of the expert but also some desired properties such as fairness.

Classic methods in constrained clustering are usually dedicated to a single type of constraint, as for instance COP-Kmeans [77] for must-link/cannot-link constraints, or [5,20] that handle the minimal cluster size constraint. Therefore they cannot be used when several types of constraint need to be integrated. On the other hand, declarative frameworks such as Integer Linear Programming (ILP), Boolean satisfiability (SAT) or Constraint Programming (CP) offer tools to express in the same setting different types of constraints, using decision variables and native constraints. Depending on the framework or on the problem formulation in this framework, the encoding of constraints may be more or less efficient (number of decision variables, number of native constraints, etc.). In this paper we present a survey of declarative methods for constrained clustering. They are based on ILP, SAT or on CP. Instead of presenting each method separately, we have identified three families of approaches, according to the way they build the clustering:

- clustering by assignment of instances that builds a partition in K clusters by assigning each instance to a cluster;
- clustering by selection of clusters that first builds a set of potential clusters and then chooses K clusters forming a partition and satisfying the constraints;
- deep constrained clustering: a general problem for distance-based clustering is the choice of the distance and therefore the design of an adequate representation space for data; deep clustering allows to learn both a representation space for data and a clustering.

We also present some methods that consider clustering in a human-in-the-loop framework: given an initial partition and some constraints, they modify the current partition to integrate new constraints.

For the two first families, we highlight their common points, standardize the decision variables and discuss the optimization criterion and the constraint encoding depending on the declarative framework. The efficiency of these methods does not depend only on the declarative framework but also on the encoding (choice of the decision variables, expression of the constraints) in the chosen framework.

The methods presented so far allow to integrate different types of constraints, guarantee to satisfy all the constraints when possible and find a global optimized solution. The methods are applied on the initial representation of the data and they do not have the ability to learn a new representation space more suited to the clustering problem under the given constraints. On the other hand, Deep Learning enables representation space learning; this explains why deep clustering methods have recently been developed, as well as deep constrained clustering methods. Most of them integrate constraints by defining a loss function for each type of constraints. But there exists a method that following the same principle as declarative approaches, allows to encapsulate different types of constraints into a single loss function, as soon as the constraints can be written in propositional logic.

The contributions of this paper are:

- An inventory of constraints for Constrained Clustering, coming from two fields of clustering, that are distance-based clustering and conceptual clustering. The constraints are organized into three categories: structural constraints at different levels (instance, cluster), semantic constraints based on features, conceptual constraints for cluster description.
- A survey of declarative constrained clustering methods. They rely on Integer Linear Programming, on Constraint Programming or on SAT and they heavily depend on the choice of decision variables to model the clustering problem and on the way constraints are implemented. We propose to organize them into two main families: whether they search for an assignment of instances to clusters or whether they search for a partition from a set of clusters. We highlight their common characteristics for modeling the clustering problem (decision variables, constraint encoding), before discussing their specific features. We also present a deep clustering method that relies on a propositional representation of constraints for computing a constrained loss.
- An illustration of the interest of declarative frameworks for descriptive clustering, which allows to integrate into a distance-based clustering problem ideas stemming from a conceptual-based setting.

The paper is organized as follows. Section 2 introduces clustering (distance-based, conceptual and constrained clustering) and declarative frameworks. Section 3 presents an inventory of constraints, coming from both distance-based and conceptual clustering domains. Section 4 provides an overview on the families of declarative approaches and points out their common points. Sections 5 and 6 are devoted to the methods that search for an assignment of instances to clusters, either for distance-based or for conceptual clustering. Section 7 presents the methods based on cluster selection and Section 8 an approach for integrating constraints in Deep Clustering architecture.

2. Preliminaries

2.1. Clustering

Given a set $\mathcal{O}$ of N data instances, clustering is a Machine Learning task that aims at finding an organization of the instances in classes or clusters, so that instances in the same cluster are similar. Clustering algorithms can be considered along three main dimensions: the structure organizing the clusters (a partition as in K-means [54], a dendrogram as in hierarchical clustering [70,26]), the definition of a cluster (distance-based /property-based) and the family of algorithms. In this review, we consider the most usual case of clustering, which aims at building a partition $\mathbb{P}$ of the data instances, where a partition is defined as a set of K clusters, $\mathbb{P} = \{C_1, \dots, C_K\}$ such that:

- $C_k \neq \emptyset$ for all clusters C_k ,
- $C_k \cap C_{k'} = \emptyset$ for each pair of different clusters C_k and $C_{k'}$,
- $\bigcup_{k \in \{1, \dots, K\}} C_k = \mathcal{O}$.

Often the number of clusters K needs to be fixed beforehand, as for instance in K-Means [54]. Some methods, e.g. density-based clustering approaches as for instance DB-Scan [36] can automatically determine the number of clusters, depending on parameters for defining the notion of density. We will present in Section 5 some declarative approaches, which require only bounds on the number of clusters to be given.

We consider two settings for clustering:

- distance-based clustering: a distance or a dissimilarity matrix between instances is either given or computed from a set of features,
- conceptual clustering: objects are described by Boolean properties (or descriptors) and a cluster must be a concept, that is a set of instances and a set of properties satisfying the condition that all the instances in the cluster and only them satisfy the associated properties and this set of properties is maximal.

Notations. In the following, we consider a set $\mathcal{O}$ of N instances, also called objects or points, $\mathcal{O} = \{o_1, \dots, o_N\}$. We distinguish the case where the instances are described by features, which may be numeric, Boolean, categorical, ... allowing the computation of a pairwise distance between instances, from the case where they are described by Boolean properties for conceptual clustering or descriptive clustering. We use the following notations:

- $[u, v]$ denotes the set of integers $\{i \mid u \leq i \leq v\}$.
- $\#A$ denotes the cardinality of a set A .
- $\mathcal{X}$ denotes a set of features with $\#\mathcal{X} = F$ and $\mathbb{X}[i, f]$ denotes the value taken by instance o_i on feature f , for $i \in [1, N]$ and $f \in [1, F]$.
- $\mathcal{B}$ denotes a set of Boolean properties with $\#\mathcal{B} = B$, and $\mathbb{B}_{N, B}$ the $N \times B$ Boolean matrix defined by $\mathbb{B}[i, p] = 1$ when instance o_i satisfies property p .
- d a pairwise distance between instances and $\mathbb{D}$ a distance or dissimilarity matrix between instances, $\mathbb{D}[i, j] = d(o_i, o_j)$.
- If $\mathbb{P}$ is a partition of K clusters, clusters are denoted by $C_1, \dots, C_K$ and for all instances o_i , $\mathbb{P}[i]$ denotes the cluster that o_i belongs to, i.e. $\mathbb{P}[i] = k$ when o_i belongs to C_k .

- In this paper, when introducing decision variables which are matrices, we use subscribe notations, i.e. for a matrix C we use C_{ij} for $C[i, j]$.

Let us note that depending on the setting, objects can be described either by $\mathcal{X}$, or by $\mathcal{B}$, or both and that in this last case $\mathcal{X}$ and $\mathcal{B}$ may not be disjoint, meaning that some features can be used both for computing a pairwise distance and for describing the clusters.

2.1.1. Conceptual clustering

Conceptual inductive learning was formalized in [57]. It was defined as, we quote, *inductive learning whose final products are symbolic expressions expressed in high-level, human-oriented terms and forms*. It was decomposed into two domains, that are learning from examples (supervised learning) and learning from observations for discovering either interesting patterns or structures in datasets (unsupervised learning). In the proposed framework, observations were described by descriptors, which could be nominal (categorical), as for instance the color of an object, linear (the domain is a totally ordered set), as for instance the weight of an object, or structured (the domain of the descriptor is structured into a tree or an oriented graph), as for instance the shape of an object whose domain could be a rectangle or a parallelogram.¹ In [59], a conceptual clustering method, called Cluster/2, was proposed. In this approach, objects are grouped into clusters that are characterized by conjunctive formula, i.e. conjunction of properties; disjunctions are allowed only for defining the values a descriptor can take, as for instance $color = (red \vee blue) \wedge shape = rectangle$, and given a predefined number K of clusters, the aim is to find K clusters with K descriptions that are disjoint. Cluster/2 was based on the notion of star where given an observation e and a set of observations E , the star of e against E was the set of all maximally general descriptions covering e and not covering any observation in E .

Cobweb [37] is an incremental conceptual learning approach. Given a set of observations described by categorical descriptors, the aim is to find a hierarchy of concepts, i.e. a tree structure where each node contains a subset of observations (a cluster or a class) and represents a probabilistic concept described by the probability of this cluster (approximated by the relative frequency of the concept in the population) and a vector that gives for each descriptor its values and their probabilities. For instance suppose a dataset of 500 instances representing persons and described by an attribute *status* with two possible values that are *professor* and *student*, a cluster containing 80 instances among 500 with 60 professors and 20 students will have a probability of 0.16 (80/500) and the descriptor *status* will have the value *professor* with probability 0.75 (60/80) and the value *student* with a probability 0.25 (20/80). It is incremental: observations are processed sequentially and the problem can be formulated as given a hierarchy of concepts and a new observation o , build a new hierarchy integrating o .

Given a new observation o , at each level of the existing hierarchy composed of a set of disjoint classes, Cobweb either inserts o in its nearest class and updates its probabilistic description to account for o or creates a new class containing only o . Cobweb is incremental and it is therefore sensitive to the order observations are presented. Two operations are added to counter the effects of previous choices that are merging two existing clusters in a single one or splitting a cluster into two clusters. The choice of the best operation, is based on the category utility measure [40] that attempts to maximize both the probability that two objects in the same class have values in common and the probability that objects in different categories will have different values. Given a concept C , a descriptor A and a value v the aim is to maximize $Pr(C|A=v)$ (predictive nature of $A=v$, i.e. ability to predict C) and to maximize the probability $Pr(A=v|C)$ (typical nature of $A=v$, i.e. ability to characterize C):

$$\frac{1}{K}(\sum_k \sum_i \sum_j Pr(A_i = v_i^j) \times Pr(C_k | A_i = v_i^j) \times Pr(A_i = v_i^j | C_k))$$

This can be written as:

$$\frac{1}{K}(\sum_k \sum_i \sum_j Pr(C_k) \times Pr(A_i = v_i^j | C_k)^2)$$

These approaches have set the basis for conceptual clustering.

As shown by these two seminal work, a key point to conceptual clustering is to be able not only to build clusters (group of instances) but also to attach a description to these clusters. Usually two properties are expected: the description should be characteristic of the cluster (instances in the cluster should satisfy it) and discriminant w.r.t. other clusters (instances in the other clusters should not satisfy it). In this paper we are interested in declarative methods for conceptual clustering and the approaches that we present are derived from [66], which first proposed a Constraint Programming approach for itemset mining and the extension to k -pattern set mining under constraints [43]. This explains why we focus on conceptual clustering based on Formal Concept Analysis. It takes as inputs a set $\mathcal{O}$ of N instances described by a set $\mathcal{B}$ of Boolean properties, as illustrated in Table 1.

A *pattern* P is defined as a set of properties. An object is *covered* by P if it satisfies all the properties of P . A *pattern* P is closed when all the objects covered by P have only the properties of P in common. A *concept* is defined as a couple (C, P) , with $C \subseteq \mathcal{O}$, $P \subseteq \mathcal{B}$ such that P is a closed pattern and C is the set of objects covered by P . If we consider Table 1, the pairs $(\{o_1, o_2\}, \{a, b\})$, $(\{o_3, o_4, o_5\}, \{b, c\})$, $(\{o_1, o_2, o_3, o_4, o_5\}, \{b\})$ represent all the concepts. The concepts can be determined by searching the closed patterns and computing the objects they cover. The number of concepts can be high: usually a minimum frequency threshold is set and only closed patterns that cover more objects than this threshold are kept.

In this context, conceptual clustering aims at finding a set of K concepts $(C_1, P_1), \dots, (C_K, P_K)$, so that $\{C_1, \dots, C_K\}$ forms a partition of $\mathcal{O}$.

¹ The description language was more complex, since it was a restricted form of predicate calculus called VL2 containing function symbols and predicates.

Table 1
Example for Formal
Concept Analysis.

	<i>a</i>	<i>b</i>	<i>c</i>
o_1	1	1	0
o_2	1	1	0
o_3	0	1	1
o_4	0	1	1
o_5	0	1	1

Table 2
Example for Formal
Concept Analysis.

	<i>a</i>	<i>b</i>	<i>c</i>
o_1	1	1	0
o_2	1	1	1
o_3	0	1	1
o_4	0	1	1
o_5	0	1	1

If we consider the example given in Table 1, then a partition could be $(\{o_1, o_2\}, \{a, b\}), (\{o_3, o_4, o_5\}, \{b, c\})$. If a minimum frequency threshold τ is set, then only concepts with at least τ objects are considered. It is not always possible to find a partition from a set of concepts. For instance if we consider the example given in Table 2, the concepts are $(\{o_2\}, \{a, b, c\}), (\{o_1, o_2\}, \{a, b\}), (\{o_2, o_3, o_4, o_5\}, \{b, c\}), (\{o_1, o_2, o_3, o_4, o_5\}, \{b\})$ and it is not possible to build a partition from them and overlapping between clusters must be allowed.

The system Ecclat [21] aims at finding a set of clusters with a minimum of overlapping between clusters. Let us recall that a concept is determined by a closed pattern. For selecting the best closed patterns, a measure of interestingness is defined on each pattern: it takes into account the concentration of the pattern that depends on the number of objects covered by this pattern that satisfy other closed patterns (and thus could be overlapping) and the homogeneity of the pattern that depends on the size of the pattern, the number of objects it covers and the number of properties of these objects not in the pattern (called the divergence).

2.1.2. Distance based clustering

Given $\mathcal{O} = \{o_1, \dots, o_N\}$ and a dissimilarity measure $d : \mathcal{O} \times \mathcal{O} \rightarrow \mathbb{R}^+$, the aim of distance-based clustering is to find a partition of $\mathcal{O}$ into K homogeneous clusters, $\mathbb{P} = \{C_1, \dots, C_K\}$, where the notion of homogeneity is usually characterized by an optimization criterion, which aims at making clusters compact and well separated. Different optimization criteria exist, the most common are below [47]:

- minimizing the sum of squared errors (called SSE or WCSS for within cluster sum of squares) defined by:

$$WCSS(\mathbb{P}) = \sum_{k \in [1, K]} \sum_{o \in C_k} d(o, \mu_k)^2$$

where μ_k denotes the center of cluster C_k . This is the criterion used in the K-means algorithm.

- minimizing the within cluster sum of distances WCSD:

$$WCSD(\mathbb{P}) = \sum_{k \in [1, K]} \frac{1}{2} \sum_{o, o' \in C_k} d(o, o')^2$$

- minimizing the maximum diameter of the clusters $diam(\mathbb{P})$: the diameter of a cluster C_k is defined as the largest distance between any two of its instances $diam(C_k) = \max_{o, o' \in C_k} d(o, o')$. The diameter, $diam(\mathbb{P})$, of partition $\mathbb{P}$ is the maximal diameter of its clusters:

$$diam(\mathbb{P}) = \max_{k \in [1, K]} diam(C_k)$$

- maximizing the minimum split (or margin) between clusters $split(\mathbb{P})$: the split/margin between two clusters $C_k, C_{k'}$ is defined as $split(C_k, C_{k'}) = \min_{o \in C_k, o' \in C_{k'}} d(o, o')$. The split, $split(\mathbb{P})$, of a partition $\mathbb{P}$ is the smallest split between any two different clusters:

$$split(\mathbb{P}) = \min_{k, k' \in [1, K], k \neq k'} split(C_k, C_{k'})$$

For the same dataset, different criteria may yield to different partitions. An example is given in Fig. 1 for a toy dataset of instances described in 2 dimensions. For applying a clustering on real data, the user must choose the criterion and this choice can have a strong impact on the result.

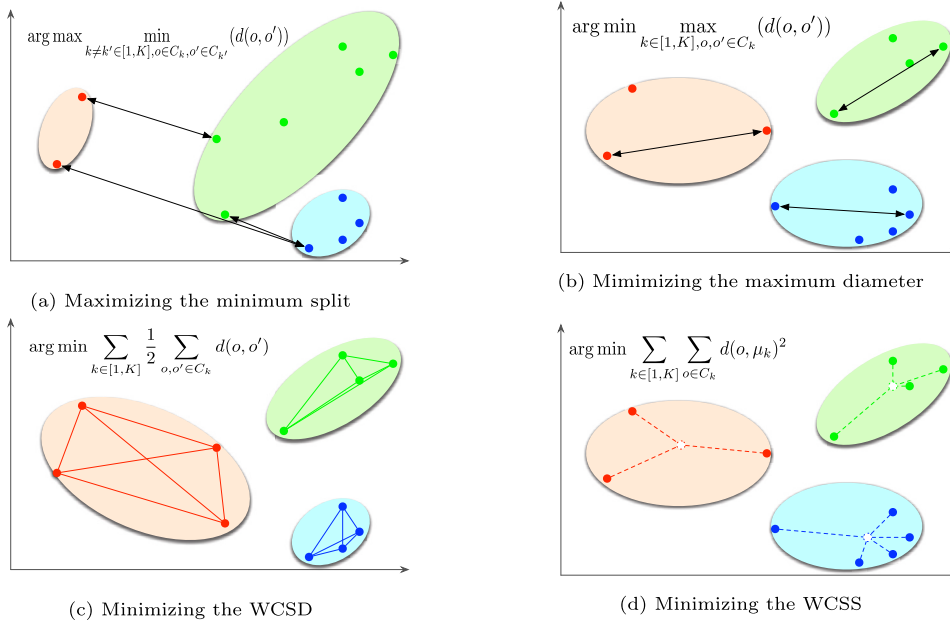


Fig. 1. For the same dataset, different criteria may yield different results.

All these criteria except the minimal split criterion are NP-Hard. Finding a partition maximizing the minimal split between clusters is Polynomial [27]. This problem however becomes NP-Hard with the presence of user constraints [33]. Concerning the maximal diameter criterion, the problem is polynomial when searching for the best partition in 2 clusters ($K = 2$), but it becomes NP-Hard as soon as searching for a partition with at least 3 clusters ($K \geq 3$) [46]. The NP-Hardness of the WCSS criterion whatever the dimension is even with $K = 2$ is shown in [1]. Because of the problem hardness, classic algorithms always search for a local optimum (e.g. K-means for WCSS criterion).

2.1.3. Comparison of the two frameworks

Distance-based clustering is the most studied approach for clustering and many methods have been developed, whereas conceptual clustering is more related to symbolic learning. They can be compared according several dimensions:

- Optimization criteria: for distance-based clustering, it is based on geometric properties of the clustering (compactness, separation between clusters, etc.) whereas in conceptual clustering, the interest is to find concepts that can be defined in intension by properties and in extension by instances.
- Distance-based clustering requires a distance (or a dissimilarity measure in some cases) to be computed. It can handled numeric features (e.g. Euclidean distance), Boolean features (Hamming or Jaccard distance). Conceptual clustering requires Boolean features to be given, and numeric values must be discretized.
- The results of distance-based clustering are difficult to interpret, since they are based on a pairwise distance, hiding the influence of each attribute, whereas conceptual clustering result is interpretable in the sense that clusters form concepts that are defined by the properties they satisfy.

2.2. Constrained clustering

From the point of view of applications, clustering is often seen as an exploratory process, aiming for domain experts at better understanding the underlying structure of data. Experts have often some *a priori* knowledge of the data. When the output partition does not fit their expectations, they can only change either the data representation (features, Boolean properties), or the learning parameters (number of clusters, optimization criterion, distance, ...)

This is difficult for an expert, requiring a fine understanding of the clustering process for preprocessing the data (feature selection, weighting, feature construction). Deep clustering, allowing the automatic building of a new data representation can alleviate the task, but at the expense of a loss of interpretability of the results.

To tackle this problem, it has been first proposed to integrate in the clustering process must-link (ML) and cannot-link (CL) constraints stating that two instances must be (respectively cannot be) in the same cluster, thus leading to semi-supervised clustering [77]. It has then been extended to integrate more general expert knowledge, written as constraints. This is called Constrained Clustering: constraints can be put on instances, as for instance the classical ML and CL constraints, or on clusters as for instance the size of the clusters. Section 3 presents a review of the different kinds of constraints that can be introduced.

Introducing constraints in a clustering algorithm makes the problem more difficult. As an illustration, maximizing the split between clusters is a polynomial criterion, which becomes NP-Hard with constraints. Many works have been developed for Constrained Clustering, mainly for ML and CL constraints. The algorithms that have been proposed can be studied along different criteria:

- whether the output partition is a local/global optimum,
- whether they are generic or designed for a specific kind of constraints,
- their ability to satisfy all the constraints,
- their ability to control the constraints that are satisfied,
- their ability to generalize the constraints known on some instances on other unconstrained instances, for instance to their neighbors,
- their complexity and scalability.

We can distinguish four families of approaches for Constrained Clustering

- Adaptation of clustering algorithms, for instance COP-Kmeans [77] or PCK-means [7] for ML/CL constraints, or [60,5,19] for cluster size constraints.
Such methods are dedicated to a specific kind of constraints. For instance, during the assignment step of K-means, COP-Kmeans [77] enforces ML/CL constraints while [5] enforces cardinality constraints (lower bounds on the size of the clusters). In this last case, this step is modeled as a Minimum Cost Flow (MCF) linear network optimization problem. These methods usually do not find the optimal solution, and even depending on the methods and on the kind of constraints, they may fail when enforcing constraints, as for instance in COP-Kmeans.
- Metric learning for ML and CL constraints, as for instance MPCK-MEANS [8], HMRF-Kmeans [4], that aims at learning a new representation space to bring closer pairs of instances linked by a ML constraint and to put away pairs of instances linked by a CL constraint [6]. These methods are heuristics and do not guarantee the satisfaction of all the constraints, but they allow learning a new metric space taking into account the constraints.
- Declarative frameworks for clustering (the scope of this paper). The constrained clustering problem is formulated in a declarative framework, such as Integer Linear Programming, Boolean satisfaction or Constraint Programming. One of the advantages of these approaches is their ability to integrate several kinds of constraints and to enforce the satisfaction of all the constraints, at the price of scalability.
- Deep Constrained Clustering. The emergence of Deep Learning has led to new deep architectures for clustering, such as the system DEC [78] that combines an auto-encoder and a self-training component improving a partition obtained by K-means. Constrained deep clustering algorithms have been developed; they usually integrate a loss function for each type of constraints as for instance in [67] or in [48] for ML/CL constraints. Let us notice that in this last work ML/CL constraints are transformed into triplet constraints and a loss for triplet constraints is then introduced. In [81] a loss is introduced for ML/CL, Triplet and size constraints. These losses are generally not specific to clustering and have been introduced in supervised learning or in metric learning (see for instance [13,71]). [61] proposes a general loss function allowing to encapsulate in a single framework constraints written in propositional logic. This work is presented in Section 8.
The use of auto-encoders allows to learn a new representation space more suited to the constraints, but deep learning does not allow to find an optimal solution and enforcing constraints through a loss does not guarantee the satisfaction of all the constraints.

2.3. Declarative paradigms

Declarative formalisms offer powerful tools to solve combinatorial problems. Declarative methods in constrained clustering use formalisms such as Integer Linear Programming (ILP), Boolean satisfiability (SAT) and Constraint Programming (CP). Using these formalisms, constrained clustering problems are modeled in a declarative way and solvers are used to find solutions for these problems.

Among these formalisms, ILP is a well known mathematical optimization setting, where a problem is formulated using linear constraints. The canonical form of an integer linear program is:

$$\begin{aligned} &\text{maximize} && \mathbf{c}^T \mathbf{x} \\ &\text{subject to} && \mathbf{A} \mathbf{x} \leq \mathbf{b} \\ &&& \mathbf{x} \geq 0 \\ &&& \mathbf{x} \in \mathbb{Z}^n \end{aligned}$$

where $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$ are vectors and $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a matrix. The variables are constrained to be integers. A variant is Mixed-Integer Linear Programming, where only a subset of variables are constrained to be integers, the other variables are allowed to be non-integers. The objective function and the constraints, except the integrality constraints are all linear. Because of the restriction that some variables are integers, solving an ILP is a NP-Hard problem [64].

Another well known formalism is Boolean satisfiability (SAT). In this formalism, the variables are Boolean (binary), and so can be *True* (1) or *False* (0). A literal is either a Boolean variable X or its negation $\neg X$. A clause is a disjunction of literals. A propositional formula is in conjunctive normal form (CNF) if it is written as a conjunction of clauses. Given a CNF formula, a SAT problem aims at determining whether there exists an interpretation of the variables, that is an assignment of variables to 0 or 1, that satisfies it.

A propositional logic formula is expressed using Boolean variables and logic connectors, that are conjunction ($\wedge$), disjunction ($\vee$), negation ($\neg$) and implication ($\rightarrow$). Using the laws of Boolean algebra, every propositional logic formula can be transformed into an equivalent CNF, and therefore a SAT problem can be defined to determine the satisfiability of a logic formula. Without using extra variables, the size of the CNF can be exponential in the worst case in the size of the original formula. However this size can be linear to the size of the original formula, using extra variables with Tseitin encoding [73].

SAT is the first problem that was proved to be NP-Complete [15]. However, a 2-SAT problem, where the number of literals in each clause is at most 2, can be solved in polynomial time. In spite of SAT complexity, efficient algorithms for solving SAT have been developed and SAT solvers can efficiently solve problems involving thousands of variables and millions of clauses. The main algorithm is the Davis-Putnam-Logemann-Loveland algorithm (DPLL) [30], which is a tree-based backtrack search procedure, enhanced with other techniques such as unit propagation or pure literal elimination [80]. Modern SAT solvers are based on DPLL and are improved by techniques such as Conflict Driven Clause Learning and non-chronological backtracking [72,49], two-watch literals scheme, restart scheme [44], etc. For more information on SAT solvers, we refer the reader to [10].

Constraint Programming (CP) is a powerful paradigm based on Artificial Intelligence or Operational Research methods to solve combinatorial problems that can be expressed by decision variables taking their values in a predefined domain and constraints expressing relations among variables. The domain of a variable can be a finite set of values or a numeric interval. A *Constraint Satisfaction Problem (CSP)* is a triple $\langle X, Dom, C \rangle$ where:

- $X = \langle x_1, x_2, \dots, x_n \rangle$ is a n -tuple of variables,
- $Dom = \langle Dom(x_1), Dom(x_2), \dots, Dom(x_n) \rangle$ is a corresponding n -tuple of domains, one for each variable,
- $C = \langle C_1, C_2, \dots, C_t \rangle$ is a t -tuple of constraints where each constraint C_i expresses a relation on a subset of X .

A solution of a CSP is a complete assignment of values from $Dom(x_i)$ to each variable x_i that satisfies all the constraints of C . A *Constraint Optimization Problem (COP)* is a CSP with an objective function to be optimized. An optimal solution of a COP is a solution of the CSP that optimizes the objective function. In general, solving a CSP is NP-Hard. Nevertheless, the methods used by the solvers enable to efficiently solve a large number of real applications. They rely on constraint propagation and search strategies.

Constraint propagation of a constraint c reduces the domain of the variables of c , by removing some or all inconsistent values, i.e., values that cannot be part of a solution of c . Different kinds of constraints are available in CP, they can be elementary constraints expressing arithmetic or logic relations, or global constraints expressing meaningful n -ary relations. One of the best known global constraints is the constraint *alldifferent*($x_1, \dots, x_n$), which imposes the variables x_i to take different values. Global constraints benefit from efficient propagation, performed by a filtering algorithm exploiting results from other domains as for instance graph theory. From a logical point of view, a global constraint is equivalent to a conjunction of elementary constraints, e.g. the constraint *alldifferent*(x_1, x_2, x_3) is equivalent to the conjunction of binary constraints $x_1 \neq x_2 \wedge x_1 \neq x_3 \wedge x_2 \neq x_3$. The interesting point is that a global constraint with its filtering algorithm has much more powerful propagation than the set of propagators of the elementary constraints. Different global constraints are developed, each one aims at exploiting more efficiently an n -ary relation.

In a CP solver, two steps, constraint propagation and branching, are repeated until a solution is found. Constraints are propagated until a stable state is reached, in which the domains of the variables are reduced as much as possible. If the domains of all the variables are reduced to singletons then a solution is found. If the domain of a variable becomes empty, then there exists no solution with the current partial assignment and the solver backtracks. In the other cases, the solver chooses a variable whose domain is not reduced to a singleton and splits its domain into different parts, thus leading to new branches in the search tree. The solver then explores each branch, activating constraint propagation since the domain of a variable has been modified. The search strategy, which defines the choice of variable or the order of the values, can be given by the programmer to guide the search space exploration.

In the context of constraint optimization problems, where the aim is not only to find a solution but to find the best solution given an optimization criterion f , a variable x_f is introduced to represent the objective function. A global optimization constraint is a global constraint modeling the criterion to optimize f . It involves the variable x_f representing the objective function and other decision variables. Each solution induces a value for f , and based on the *branch-and-bound* mechanism for optimization, the global optimization constraint exploits this value to restrict not only the domain of x_f but also other decision variables inside the constraint. The first filtering algorithm for this kind of global constraints is proposed in [38]. A well-known example of extension of global constraints to optimization constraints is the constraint *cost_gcc* [65], which extends the Global Cardinality Constraint with cost. For more details on global constraints, search and more generally on CP, we refer the reader to [68].

Several surveys on constrained clustering have been presented. An analysis of clustering with pairwise constraints and distance-based clustering methods has been given in [18]. A comparative survey of constrained clustering methods for time-series can be found in [53]. Recently a survey on clustering methods and constraints in constrained clusters has been presented in [39]. These surveys however do not present conceptual clustering, they do not focus on declarative constrained clustering methods and do not analyse the integration of constraints in an uniform way. In our work, we do an inventory of constraints coming from distance-based and conceptual clustering and present the constraints in an uniform setting. We present a survey of declarative constrained clustering methods, by organizing them in two families, and highlight the common aspects of each family as well as specific features of each method.

3. Constraints in constrained clustering

This section presents a review of the main constraints used in Constrained Clustering. We present not only the classic constraints used in distance-based clustering but we consider also more semantic constraints, easier to set for an expert, and conceptual constraints allowing to express conceptual clustering or descriptive clustering.

3.1. Structural constraints

The constraints that are classically used are put on the structure of the partition. They express either relations between instances or required properties on the clusters.

3.1.1. Instance-level constraints

- *pairwise constraint - triplet constraint:*

Pairwise constraints have first been introduced in semi-supervised learning while triplet constraints have been introduced for metric learning. A must-link (resp. cannot-link) constraint between two instances a and b states that these instances must be (resp. cannot be) in the same cluster [76]. They are denoted by $Must_Link(a, b)$, resp. $Cannot_Link(a, b)$. A triplet constraint, denoted in the following by $ap|n$ means that a is more similar to p than to n [51]. This definition is well suited to metric learning: when searching for a new representation g of data, this constraint is usually transformed into a loss function comparing the distance between $g(a)$ and $g(p)$ to the distance between $g(a)$ and $g(n)$. It has been interpreted in terms of clustering as *if a and n are put in the same cluster then a and p must be put in the same cluster*:

$$Together(a, n) \rightarrow Must_Link(a, p)$$

where $Together$ is a predicate verifying whether two points are in the same cluster.²

- *group constraint:* the instances of a given group G must be in the same cluster: $must_group(G)$. This constraint can be expressed using Must-Link constraints, it is however more expressive by offering a more condensed form.
- *density constraint:* requires for each instance to have in its neighborhood at least m instances belonging to the same cluster as itself [32]:

$$\forall o \in \mathcal{O} \quad \#(\mathcal{N}_\epsilon(o) \cap C_{\mathbb{P}[o]}) \geq m$$

where $\mathcal{N}_\epsilon(o)$ denotes the neighborhood of o and $\mathbb{P}[o]$ the number of the cluster that o belongs to in partition $\mathbb{P}$. This requires the neighborhood to be defined, generally by $\mathcal{N}_\epsilon(o) = \{o' | d(o, o') \leq \epsilon\}$, given a radius parameter ϵ .

In [62], individual fairness [28] which requires that individuals who are close together should be grouped together has been modeled by a constraint, expressing that each instance i must be in the same cluster as a given proportion of the elements in its neighborhood $\mathcal{N}_i$.

3.1.2. Cluster-level constraints

Cluster-level constraints can be divided into 2 categories: intra-cluster constraints that express conditions on each cluster individually and inter-cluster constraints involving several clusters. They can be:

- *cardinality constraints:* either put bounds (min, max or both) on the number of instances in each cluster [5]:

$$\forall k \in [1, K], \quad \alpha \leq \#C_k \leq \beta$$

or require the size of the clusters to be balanced, by choosing appropriate values of α and β .

- *geometric constraints* [32]:

– *maximal diameter constraint:* the diameter of each cluster must be less than a predefined value δ , formalized by

$$\forall k \in [1, K], \quad diam(C_k) \leq \delta$$

– *minimal split constraint:* any two clusters C_k and $C_{k'}$ must be separated by at least a predefined value γ , formalized by

$$\forall k, k' \in [1, K], k < k', \quad split(C_k, C_{k'}) \geq \gamma$$

- *span-limited constraint:* given a subset G of instances ($G \subseteq \mathcal{O}$), limits the number of possible clusters to which elements of G can be assigned to [62]:

$$\#\{k \in [1, K] \mid C_k \cap G \neq \emptyset\} \leq m$$

² Let us notice the difference between $Together(a, b)$ and $Must_Link(a, b)$: the former is a predicate which is True when a and b are in the same cluster and False otherwise, the latter is a constraint restricting that a and b must be together. The same difference lays between $Apart$ and $Cannot_Link$.

As already stated, there exist two main approaches for clustering: the first one searches for a partition of the instances by assigning them to clusters while another one takes as input a set of cluster candidates and creates a partition by selecting clusters from the candidates. These approaches will be discussed in Section 4. In the second approach, constraints on the clustering itself can be expressed, some of them can relax the partition requirement, as below [58]:

- *cluster-overlap constraint*: putting a bound on the number of clusters each instance can belong to. In case of a partition, this bound is set to 1. If the bound is strictly larger than 1, this constraint relaxes the partition requirement since it allows cluster overlapping.
- *completeness partition constraint* gives bounds on the number of instances that are set in the partition: it covers at least ϕ instances and at most ψ instances. This constraint also relaxes the partition requirement since it allows some data instances not to be included in the final result. A complete partition must include all the instances, i.e. $\phi = \psi = N$.

In a human-in-the-loop context, finding a clustering is seen as an iterative process where once a partition of the data has been built, experts suggest constraints and the aim is to find a new partition close to the previous one while satisfying the constraints. This will be presented in Section 5. User constraints can be expressed to modify the partition, possibly changing the number of clusters [52].

- *delete cluster constraint*: this constraint deletes one cluster and its instances have to be put in the other clusters.
- *merge clusters constraint*: this constraint merges two clusters into a single one,
- *keep cluster constraint*: this constraint requires keeping a cluster (that is, having at least a point in the cluster).

3.2. Semantic constraints

Geometric constraints as well as density constraints are not easy to set for experts. Truly they are based on the distance between objects: such a distance encapsulates in a single measure different features, which are much more meaningful than the distance. To overcome this problem, semantic constraints involving the features have been introduced in [35]. They are comparable to the constraints introduced in 3.1 but they are defined on features rather than on distances. To illustrate them, let us consider that we are applying clustering on a dataset composed of people at the university, they can be student, teacher or administrative staff, students can be at a bachelor or at a master level.

- *count constraint*:
 - *property cardinality constraint*: to put bounds on the number of instances satisfying a given property p in each cluster. Let $\mathcal{O}_p$ be the set of objects satisfying p .

$$\forall k \in [1, K], \quad \alpha \leq \#(C_k \cap \mathcal{O}_p) \leq \beta$$

This can be used to express a constraint as *each group must contain at least a teacher and twenty students*.

This bound can be relative (this constraint has been called balanced binary features in [52]):

$$\forall k \in [1, K], \quad \alpha \cdot \#C_k \leq \#(C_k \cap \mathcal{O}_p) \leq \beta \cdot \#C_k$$

- *property ratio constraint*: to express a relation between the number of objects satisfying different properties in each cluster. Let $\mathcal{O}_{p_1}$ (resp. $\mathcal{O}_{p_2}$) be the set of objects satisfying p_1 (resp. p_2).

$$\forall k \in [1, K],$$

$$\alpha \cdot \#(C_k \cap \mathcal{O}_{p_2}) \leq \#(C_k \cap \mathcal{O}_{p_1}) \leq \beta \cdot \#(C_k \cap \mathcal{O}_{p_2})$$

For instance, *in each cluster, the number of teachers must be no less than half the number of students*.

A similar constraint has been used in [62] to model group-level fairness [14] requiring that in each cluster, the ratio of instances sharing a property (for instance gender) is approximately the same as the ratio in the whole dataset.

$$\forall k \in [1, K], \quad R - \epsilon \leq \frac{\#(C_k \cap \mathcal{O}_p)}{\#(C_k \cap (\mathcal{O} \setminus \mathcal{O}_p))} \leq R + \epsilon$$

with $R = \frac{\#\mathcal{O}_p}{\#\mathcal{O} \setminus \mathcal{O}_p}$ representing the ratio of the property p on the whole population.

- *geometric constraint on a feature f* : it sets bounds on numeric features inside each cluster or between the clusters [52].
 - *feature-based diameter constraint*: let $\mathbb{D}'$ a $K \times F$ matrix, specifying the expected diameter of the k -th cluster along feature f , the feature-based diameter constraint is then

$$\forall k \in [1, K], \forall f \in [1, F], \quad \text{diam}_f(C_k) \leq \mathbb{D}'[k, f]$$

where the diameter of a cluster C_k along the feature f is defined by $\text{diam}_f(C_k) = \max_{o_i, o_j \in C_k} |f(o_i) - f(o_j)|$ where $f(o_i)$ denotes the value of o_i on feature f .

Instead of giving a matrix $\mathbb{D}'$, specifying the expected diameter of the k -th cluster along feature f , the experts can choose a feature f and provide the same bound δ for all the clusters leading to a special case:

$$\forall k \in [1, K], \text{diam}_f(C_k) \leq \delta$$

- *feature-based split constraint*: Let $\mathbb{S}'$ a $K \times K \times F$ matrix, specifying the expected split between cluster k and cluster k' along feature f , the feature-based split constraint is then

$$\forall k \in [1, K], \forall k' \in [1, K], k \neq k', \forall f \in [1, F],$$

$$\text{split}_f(C_k, C_{k'}) \leq \mathbb{S}'[k, k', f]$$

where the split between two different clusters C_k and $C_{k'}$ ($k \neq k'$) is defined by: $\text{split}_f(C_k, C_{k'}) = \min |o_i \in C_k, o_j \in C_{k'} | f(o_i) - f(o_j)|$.

In the same way as for diameter, instead of giving a matrix $\mathbb{S}'$, specifying the expected split between clusters, the experts can choose a feature f and provide the same value γ for all the clusters leading to:

$$\forall k, k' \in [1, K], k \neq k', \text{split}_f(C_k, C_{k'}) \geq \gamma$$

For instance, *any two clusters must be separated by at least 2 on the age property or the ranges of age on clusters k and k' should not overlap*.

- *averaged intra-cluster constraint* let $\mathbb{M}'$ a $K \times F$ matrix, specifying the expected average of the k -th cluster along feature f , the feature-based averaged constraint is then

$$\forall k \in [1, K], \forall f \in [1, F], \text{average}_f(C_k) \leq \mathbb{M}'[k, f]$$

- *density constraint*, expressing some constraints on the neighborhood of instances, defined in terms of shared features [35]. For instance, *each student should have in his/her cluster more than 5 persons sharing the same hobby*. Given a property *prop*,

$$\forall o_i \in \mathcal{O}, \#(\{o_j \in \mathcal{O} \mid \text{prop}(o_j) = \text{prop}(o_i)\} \cap C_{\mathbb{P}[i]}) \geq m$$

where $\mathbb{P}[i]$ denotes the number of the cluster that o_i belongs to in partition $\mathbb{P}$.

- *complex logic constraints* [35], expressed by logic formulae, as for instance
 - to express conditional properties on the pairwise relations between instances: when instances are put together or apart in clusters then other instances must be or cannot be in the same cluster

$$\wedge_{(o_i, o_j)} \text{reln}(o_i, o_j) \rightarrow \wedge_{(o_u, o_v)} PW(o_u, o_v)$$

where *reln* is either *Together* (instances are put in the same cluster) or *Apart* (instances are put in different clusters) and *PW* represents either a ML or a CL constraint. For instance, *if instances 1 and 15 are put in the same cluster, then instances 3 and 9 must be in different clusters*. This can be generalized to *complex logic group constraint* that expresses a relation on groups of instances, as for instance

$$\text{grouped}(G_1) \wedge \neg \text{grouped}(G_2) \rightarrow \text{must_group}(G_3)$$

where *grouped*(G) is a predicate that is satisfied if all the instances in G are in the same cluster.

- to express conditional properties on the clusters, as for instance *any cluster having more than 2 professors must have at least 10 PhD students*.

$$\forall k \in [1, K] \text{ satisfy}(\text{Prop}_1, C_k) \rightarrow \text{satisfy}(\text{Prop}_2, C_k)$$

where Prop_i represented a property put on a cluster.

It is important to notice that some of these constraints can be translated into pairwise constraints and therefore could be easily handled by semi-supervised clustering methods. For instance, a diameter constraint requesting that the diameter should not exceed γ can be expressed by cannot-link constraints on any two instances o_i, o_j with $d(o_i, o_j) > \gamma$.

3.3. Conceptual constraints

We suppose that we have N instances described by a set $\mathcal{B}$ of B Boolean properties. Let $\mathbb{B}_{N,B}$ defined by $\mathbb{B}[i, p]$ is true when the i -th instance i satisfies the p -th property. The aim of conceptual clustering is to find K concepts $(C_1, P_1), \dots, (C_K, P_K)$ so that $C_1, \dots, C_K$ forms a partition of $\mathcal{O}$. Enforcing each cluster to be a concept can be expressed using constraints, which we call conceptual constraints. Let (C, P) denote a pair composed of a set of objects and a pattern. Coverage and closedness constraints [66] allow to model conceptual clustering: the output of a conceptual clustering algorithm must be a set of clusters, possibly a partition, that satisfies these constraints, and it is implemented as such in declarative approaches for conceptual clustering (see Section 6).

- *coverage constraint*: $\text{coverage}(C, P)$ expresses that C is the set of instances covered by P , it is defined as follows:

$$\forall o \in \mathcal{O}, (o \in C \leftrightarrow \forall i \in P, \mathbb{B}[o, i] = 1)$$

- *closedness constraint*: $\text{closed}(C, P)$ expresses that (1) the pattern P is closed, that is all the objects covered by P have only the descriptors of P in common and (2) C is the set of objects covered by P . It is defined as follows:

$$\forall i \in B, (i \in P \leftrightarrow \forall o \in C, \mathbb{B}[o, i] = 1)$$

Nevertheless there may not exist a partition satisfying these requirements (see for instance the example given in Table 2) and such constraints can be relaxed. For instance descriptive clustering has been proposed in [29]. This setting extends distance-based clustering by searching not only a partition of the objects based on a pairwise distance but also a description of each cluster. More precisely the instances are described by a set $\mathcal{X}$ of features and a set $\mathcal{B}$ of B Boolean descriptors. From the features, a pairwise distance d between the objects is computed. The problem is to build a partition of data based on d and a description of its clusters, defined as a subset of $\mathcal{B}$. The closedness constraint introduced in conceptual clustering is too strong, when looking only for a description of the clusters and new constraints have been introduced. Let $\mathbb{P} = \{(C_1, P_1), \dots, (C_K, P_K)\}$ denotes the partition where each cluster C_i is coupled with a pattern P_i representing the description of C_i . Description constraints are [29]:

- *non empty description*: each cluster is described by a non empty subset of descriptors.

$$\forall k \in [1, K], \#P_k \geq 1$$

- *description constraint*: an instance in a cluster C must satisfy *most* of its descriptions P (up to α exceptions, possibly $\alpha = 0$):

$$\forall o \in \mathcal{O}, (o \in C \rightarrow \#\{i \in P \mid \mathbb{B}[o, i] = 0\} \leq \alpha)$$

Let us notice that this is only an implication ($\rightarrow$). This means that when an object belongs to C , the number of properties of C it does not satisfy must be less than α . These properties are characteristic of the cluster C but not discriminant, that is an object satisfying these properties may not belong to the cluster C . An equivalence ($\leftrightarrow$) would enforce all the objects satisfying this property to belong to cluster C ; this point makes a difference between descriptive clustering and conceptual clustering.

- *almost-closed constraint*: a Boolean property is included in a cluster description P if and only if *most* of the instances in the cluster (up to β exceptions, possibly $\beta = 0$ and in this case, it is the closedness constraint) satisfy it:

$$\forall i \in B, (i \in P \leftrightarrow \#\{o \in C \mid \mathbb{B}[o, i] = 0\} \leq \beta)$$

This constraint is similar to the closedness constraint. In a concept, the set of properties is maximal and this constraint enforces the set of descriptors to be maximal. This explains why an equivalence is used here instead of an implication as in the description constraint.

Descriptive clustering can then be formalized by means of descriptive constraints that are combined with classic or semantic constraints, as introduced in Subsections 3.1 and 3.2. Descriptive clustering aims at finding a clustering with the clusters both homogeneous using a homogeneity criterion based on the features of $\mathcal{X}$ and descriptive using a descriptiveness criterion based on the descriptors of $\mathcal{B}$. The setting will be presented in Section 6.

4. Families of declarative approaches for constrained clustering

Existing declarative approaches for constrained clustering use the same principle: the clustering problem is formulated in a declarative paradigm (SAT, CP or ILP), and the solver finds a solution. A search strategy can be given to guide the solver, for instance in CP. According to our point of view, the methods can be classified into two categories, depending on the way the partition is defined: by assigning the instances to the clusters, or by selecting the clusters to compose the partition. We give below a summary of the categories, with links to the sections describing each method.

4.1. Constructing a partition by assigning the instances to the clusters

Methods in this category aim at finding an assignment of each instance to a cluster. Depending on the used formalism, the problem can be expressed differently, however these methods have the following common points on the choice of decision variables and the use of native constraints.

- *Decision variables*: for each instance $o_i \in \mathcal{O}$, one or several variables are defined to represent the cluster the instance is assigned to. Depending on the method the variables can be either integer variables or binary variables:
 - integer decision variables $G_i \in [1, K]$, where $G_i = k$ means o_i is assigned to the cluster C_k . With this type of variables, N integer variables are necessary for N objects. Since in a solution, each variable G_i will take a value $k \in [1, K]$, the instance o_i will be assigned to exactly one cluster. Therefore the fact that each instance belongs to exactly one cluster is ensured by the choice of the variables.

- Boolean (binary) decision variables $C_{ik} \in \{0, 1\}$, where $C_{ik} = 1$ means o_i is assigned to the cluster C_k . With this type of variables, $N \times K$ variables are necessary to define the partition. The requirements that each instance must be in one cluster must be enforced by partition constraints. This choice of variables allows to express relaxed conditions where one could accept some instances not to be in a cluster, or accept overlapping clusters.
- Partition and user-defined constraints: SAT/ILP/CP native constraints provided by the solver are used to express partition requirements (non-empty cluster, K clusters, disjoint clusters) and to express user-defined constraints (must-link, cannot-link, etc.).
- Clustering quality: the clustering quality representing the desired homogeneity, as for instance minimizing the maximum cluster diameter, or minimizing WCSS, etc., is expressed by the objective function associated to the model. In SAT paradigm, where there is no objective function, the quality is achieved by another mechanism, such as a binary search algorithm that calls SAT solver (Section 5.1).

Methods in this category for distance-based clustering will be presented in Section 5 and for conceptual clustering in Section 6. They use either one type of variables or both of them. For instance, for descriptive clustering, CP-based method [29] combines both types of variables to further exploit the efficiency constraints on them.

4.2. Constructing a partition by selecting clusters

Methods in this category construct a partition by selecting clusters from a set C of cluster candidates. The cluster candidates can be generated once at the beginning or on the fly. The methods have been developed using CP or ILP, either for conceptual clustering or for distance-based clustering. They all share the common points below.

- Decision variables: for each cluster candidate $c \in C$, one Boolean variable $X_c \in \{0, 1\}$ is used, where $X_c = 1$ means the cluster c is selected in the partition. The number of decision variables is therefore the number of cluster candidates.
- Partition constraints are expressed using ILP/CP native constraints. Partition constraints require that each object is assigned to one cluster and there are K clusters in the final partition. Non-empty cluster constraint is directly satisfied by ensuring $\emptyset \notin C$.
- User-defined constraints: must-link/cannot-link constraints or constraints on individual clusters (i.e. maximal size) are enforced to rule out all the cluster candidates that do not satisfy the constraints. Hence they do not further need to be expressed in the model.
- Clustering quality: the quality of each cluster candidate can be computed beforehand, since the cluster candidates are known. The clustering quality is represented by the objective function, which aggregates the individual quality of the selected clusters.

Because the number of decision variables is the number of cluster candidates, this category of methods is not realistic if any cluster could be a cluster candidate, since the number of possible clusters is 2^N for N instances. This becomes however possible in conceptual setting, where the clusters correspond to frequent or closed patterns. The patterns can be generated by specific algorithms such as Apriori [3] or LCM [74]. In distance-based clustering, ILP based methods have been developed with column generation to generate on the fly new cluster candidates. These methods will be reviewed in Section 7.

One interest of this category is that constraints on the clustering itself can be easily expressed, for instance conditions on the selection of some clusters based on the selection of some others. Another interest is that instance-level and cluster-level constraints do not need to be expressed, they are directly used to remove from the set of cluster candidates all clusters that do not satisfy one of these constraints.

A summary comparing the two categories is presented in Table 3.

5. Distance-based clustering by assignment of instances

The aim of clustering is to find a partition of homogeneous clusters. In distance-based clustering, the homogeneity is based on a distance function between pairs of instances. In this section, we survey declarative methods for distance-based constrained clustering that searches for a partition by assigning the instances to the clusters, while satisfying the constraints. They are developed relying on SAT, on ILP or on CP frameworks.

The common points of these methods are below.

- Variables: SAT/CP/ILP variables are used to express the assignment of points to clusters. In the following, SAT and ILP methods use binary variables while CP methods use integer variables.
- Partition constraints: this type of constraints expresses that the assignment of all the instances must define a partition of (i) K clusters, such that (ii) the clusters are not empty, (iii) the clusters are pairwise disjoint and (iv) all the instances are assigned. Depending on the choice of variables, some of these constraints are already satisfied.
- User-defined constraints: user-defined constraints, as for instance must-link/cannot-link, are expressed by native SAT/CP/ILP constraints.

Table 3
Comparison between the two categories.

	Assignment of points to clusters	Cluster selection
Decision variables	expressing point assignment $G_i \in [1, K]$, for $i \in [1, N]$, or $C_{ik} \in \{0, 1\}$, for $i \in [1, N], k \in [1, K]$	expressing cluster selection $X_c \in \{0, 1\}$, for $c \in [1, \#C]$ with C the set of all cluster candidates
Number of variables	$O(N)$ for G , $O(N \times K)$ for C	$O(2^N)$ in the worst case, this number is reduced in conceptual clustering by restricting the set C of cluster candidates
Framework	SAT ($K = 2$)/ILP/CP	ILP/CP
Partition requirement	expressed by SAT/ILP/CP native constraints	expressed by ILP/CP native constraints
User-defined constraints	expressed by SAT/ILP/CP native constraints	instance-level and intra cluster-level constraints are not expressed but used to removed unsatisfying cluster candidates inter cluster-level constraints are expressed by ILP/CP native constraints
Clustering quality	objective function in ILP global optimization constraint in CP	objective function in ILP global optimization constraint in CP
See	Section 5 (distance-based clustering) Section 6 (conceptual clustering)	Section 7 (distance-based and conceptual clustering)

5.1. A SAT-based method

[34] proposes a SAT-based method for finding a clustering composed of 2 clusters ($K = 2$). One can notice that in a partition of 2 clusters, clusters can be numbered by 0 and 1, therefore, for each instance i , a Boolean variable G_i will naturally represent the assignment of i : instance o_i is assigned to the cluster number G_i . Having N instances, N Boolean variables $G_1, \dots, G_N$ are introduced, which directly define a partition. Using these variables, must-link and cannot-link constraints are encoded by 2-SAT clauses as follows:

$$\text{must-link}(i, j) \equiv (G_i \wedge G_j) \vee (\neg G_i \wedge \neg G_j) \equiv (G_i \vee \neg G_j) \wedge (\neg G_i \vee G_j)$$

$$\text{cannot-link}(i, j) \equiv (G_i \wedge \neg G_j) \vee (\neg G_i \wedge G_j) \equiv (G_i \vee G_j) \wedge (\neg G_i \vee \neg G_j)$$

Using must-link and cannot-link constraints, a diameter constraint, $D \leq \alpha$, is ensured by adding a cannot-link(i, j) for all $1 \leq i < j \leq N$ such that $d(o_i, o_j) > \alpha$. In a similar way a split constraint $S \geq \beta$ is expressed by must-link constraints.

Finding a 2-cluster partition satisfying user-constraints is encoded by a 2-SAT problem $\mathbf{P}$. In order to find a partition $\mathbb{P}$ that minimizes the maximum cluster diameter $\text{diam}(\mathbb{P})$, represented by a variable D , a binary search algorithm has been proposed. With the observation that the maximum diameter D is one of the values $d(o_i, o_j)$ with $i, j \in [1, N]$, the algorithm starts by sorting all the distinct values $d(o_i, o_j)$ in increasing order, then sets the upper bound U of D to the largest value, the lower bound L to the smallest value and repeats at each step the following while $U \neq L$:

- choose α the median value of L and U and solve the 2-SAT problem $\mathbf{P}$ with the constraint $D \leq \alpha$,
- if $\mathbf{P}$ is satisfiable then the upper bound U is set to α ,
- otherwise the lower bound L is set to α .

The complexity of the algorithm for solving a 2-SAT problem $\mathbf{P}$ is $O(N^2)$, therefore the complexity of the optimization in the worst case is $O(N^2 \log(N))$.

5.2. CP-based methods

[22,25] has proposed a declarative framework using CP for constrained clustering. This framework allows to solve clustering problem with different objective functions and can integrate different types of constraints, such as must-link, cannot-link and other structural constraints.

The framework proposed in [22] has been improved in [25] by modifying the model and by developing dedicated propagation algorithms for each optimisation criterion. In this model, the value of the number of clusters K does not need to be fixed beforehand, only bounds are required, i.e. $K_{\min} \leq K \leq K_{\max}$. The clusters in a partition of K clusters are numbered from 1 to K . In order to express the cluster assignment, a variable G_i is used for each instance $o_i \in \mathcal{O}$, with $G_i \in \{1, \dots, K_{\max}\}$. Float value variables are also used to represent the value of the objective function: D for the maximum cluster diameter, S for the minimum split between clusters, W for the within-cluster sum of dissimilarity (WCSD) and V for the WCSS value. The model can be decomposed into three

components to express: the constraint that the output must be a partition, user-constraints on clustering and the objective function. These constraints are expressed using CP-constraints.

Expressing a partition. The variables G_i naturally express a partition, since the value assigned to each G_i gives the cluster that contains the point o_i . Nevertheless, several assignments of the variables $G_1, \dots, G_N$ can correspond to the same partition, for instance by interchanging the numbers of two clusters. In order to break this kind of symmetries, the clusters are numbered such that the number 1 is the index of the first created cluster and a new number $k > 1$ is used if and only if the number $k - 1$ has been already used. A straightforward way to express this condition is by using a constraint $G_1 = 1$ and the constraints $G_i \leq \max_{j \in [1, i-1]} (G_j) + 1$. However, in order to have better interactions and propagations between these relations, a better way is to sum up these relations into one global constraint with a good filtering algorithm. The CP-constraint `precede` ($[G_1, \dots, G_N], [1, \dots, K_{\max}]$) is therefore used. This constraint ensures that

- $G_1 = 1$
- and moreover, if $G_i = k$ with $1 < k \leq K_{\max}$, there must exist at least an index $j < i$ such that $G_j = k - 1$.

The requirement to have at least $K_{\min}$ clusters means that all the numbers among 1 and $K_{\min}$ must be used in the assignment of the variables G_i . When using the constraint `precede`, one only needs to require that at least one variable G_i is equal to $K_{\min}$. This is expressed by using the CP-constraint `atleast` ($[G_1, \dots, G_N], K_{\min}, 1$), which means $\#\{i \mid G_i = K_{\min}\} \geq 1$. Therefore, partition requirement is enforced using these two CP-constraints:

```
precede([G1, ..., GN], [1, ..., Kmax])
atleast([G1, ..., GN], Kmin, 1)
```

Expressing the objective function. Different optimization criteria are available: minimizing the maximum diameter D of the clusters, maximizing the minimum split S between clusters, minimizing the within-cluster sum of dissimilarities (WCSD) or minimizing the within-cluster sum of squares (WCSS). CP constraints can be used to directly express these criteria. For instance, in [22], when minimizing the maximum diameter, since the variable D represents the maximum diameter, any two instances at a distance greater than D must be in different clusters:

$$\forall i < j \in [1, N], (d(o_i, o_j) > D) \rightarrow (G_i \neq G_j).$$

Since D is a variable and its value is still unknown, these relations are expressed using reified constraints.³ However, in order to have better constraint propagations, which means a more efficient model, a global optimization constraint has been developed for each criterion. For instance, if the user chooses to optimize the sum of squares, the variable V will be linked by the constraint `WCSS` ($[G_1, \dots, G_N], V, d$) and the objective function will be minimize V . Global optimization constraints have been therefore developed for the diameter or the split criterion [25], WCSD [23] and WCSS [24] and for each constraint a filtering algorithm has been developed.

The worst case complexity of the filtering algorithm for the diameter, the split or the WCSD constraints is $O(N^2)$. For the WCSS constraint, the worst case complexity is $O(Kq^2 \log q + qN)$, where q is the number of unassigned variables among the decision variables $G_1, \dots, G_N$ ($q \leq N$).

In order to improve the performance of CP solvers, different search strategies have been elaborated adapted to each criterion. The work done in [25] illustrates that the development of a clustering method based on CP is not only on modeling the problem using CP constraints, but also on developing global constraints to further exploit the meaning of the objective function, and on designing search strategy to improve the efficiency.

Concerning the famous WCSS criterion, which is used in K-means algorithm, a better performing method for unconstrained exact clustering is the Repetitive Branch-and-Bound Algorithm (RBBA) [11]. A CP-based framework that extends RBBA using CP to support user constraints has been developed for constrained clustering for the WCSS criterion [41].

Expressing user constraints. All popular user-defined constraints may be straightforwardly integrated. Some examples are below.

- Must-link constraint: a must-link constraint on two instances i, j is expressed by: $G_i = G_j$.
- Cannot-link constraint: a cannot-link constraint on i, j is expressed by: $G_i \neq G_j$.
- Minimal size α of clusters: this means that each instance must be in a cluster with at least α instances (including itself). This means for each $i \in [1, N]$, the assigned value of the variable G_i must appear at least α times in the array $[G_1, \dots, G_N]$. This is ensured by using the CP-constraint `atleast` ($[G_1, \dots, G_N], G_i, \alpha$) for each $i \in [1, N]$.
- Maximal size β of clusters: each number $c \in [1, K_{\max}]$ must appear in the array $[G_1, \dots, G_N]$ at most β times (this is still true for an unused value $c \in [K_{\min} + 1, K_{\max}]$, since it appears 0 time). Therefore, for each $c \in [1, K_{\max}]$, the CP constraint `atmost` ($[G_1, \dots, G_N], c, \beta$) is put, which expresses $\#\{i \mid G_i = c\} \leq \beta$. Let us note that differently from the way of encoding min-

³ A reified constraint is a logical constraint of the form $A \rightarrow B$ or $A \leftrightarrow B$, where A and B are also constraints. The reified constraint $A \rightarrow B$ means that the constraint B must be satisfied if A is satisfied, and $\neg A$ must be satisfied if $\neg B$ is satisfied.

imal size constraint, which needs N at least constraints, the encoding of maximal size constraint needs only K_{max} at most constraints.

- Maximum diameter constraint: the diameter of each cluster must not exceed γ . This constraint can be expressed by putting one constraint $G_i \neq G_j$ for each couple $i, j \in [1, N], i \neq j$ such that $d(o_i, o_j) > \gamma$. The number of constraint is of order N^2 . By exploiting the developed global constraint $\text{diameter}([G_1, \dots, G_N], D, d)$, which enforces that D is the maximum diameter of the clusters, one only needs to put $D \leq \gamma$ together with the global constraint. This information is integrated to the global constraint to yield constraint propagation.
- Density constraint: a classic density constraint expresses that each point must have in its neighborhood of radius ϵ , at least m points belonging to the same cluster as itself. For each $i \in [1, N]$, let $\mathcal{N}_{i\epsilon}$ be the set of points in its ϵ -neighborhood $\mathcal{N}_{i\epsilon} = \{j \mid d(o_i, o_j) \leq \epsilon\}$. This constraint can be expressed using the constraint: $\text{atleast}([G_j \mid j \in \mathcal{N}_{i\epsilon}], G_i, m)$. This encoding can be used to express density constraints where the neighborhood of a point is defined using some features, as for instance “Each person should have in her group more than 5 persons sharing the same hobby” [35]. The set of the neighborhood $\mathcal{N}_i$ is therefore $\{j \in [1, N] \mid \text{hobby}(i) = \text{hobby}(j)\}$, and the constraint can be expressed by $\text{atleast}([G_j \mid j \in \mathcal{N}_i], G_i, 5)$.
- Complex logic constraints: Boolean connectors can be used in CP constraints, which enable encoding complex logic user constraints, as the following examples [35].
 - Two instances 3, 9 are in the same cluster if the instances 11, 15 are in different clusters. Two Boolean variables B_1, B_2 are introduced with the constraints: $B_1 \leftrightarrow (G_{11} \neq G_{15})$, $B_2 \leftrightarrow (G_3 = G_9)$ and $B_1 \leq B_2$.
 - Any group having more than 3 professors must have more than 5 PhD students. For each $k \in [1, K]$, let P_k and S_k represents the number of professors and PhD students in the cluster k , the constraint can be expressed by:

$$\begin{aligned} &\text{exactly}([G_i \mid i \in [1, N], \text{professor}(i) = \text{True}], k, P_k) \\ &\text{exactly}([G_i \mid i \in [1, N], \text{student}(i) = \text{True}], k, S_k) \\ &P_k > 3 \rightarrow S_k > 5 \end{aligned}$$

CP models use therefore N integer decision variables and two CP constraints to express that the output must be a partition. Depending on the type of user constraint, the number of CP constraints to express it can be a constant, as for instance for must-link/cannot-link constraints or for a maximal size constraint, or it can be N for a density constraint. A global optimization constraint has been developed for each criterion, as for instance the maximal diameter or the WCSS, to have a more efficient filtering algorithm.

5.3. ILP-based methods

A related problem to constrained clustering has been studied in [52,62]. Let us suppose that a partition has already been built which is in general good but with a few undesirable properties. For instance after clustering data from the university, some clusters have too many students compared to the number of teachers, or the range of ages for some clusters is too large. New constraints must then be introduced to get a better solution but restarting a clustering process including these new constraints can lead to a quite different clustering from the initial one. Two different settings have been considered:

1. *minimal clustering modification problem under constraints*: given a partition $\mathbb{P}$, that is a hard assignment of points to clusters and a set of constraints $\mathbb{C}$, the aim is to find a new partition satisfying the constraints and minimizing the number of changes between the two partitions,
2. *partition building from a soft assignment under constraints*: given a soft assignment $S_{N \times K}$ of points to clusters and a set of constraints $\mathbb{C}$, find the best partition according to S satisfying all the constraints.

5.3.1. Minimal clustering modification problem

Given a partition $\mathbb{P}$ and a set of constraints $\mathbb{C}$, the problem is to find a new partition $\mathbb{P}'$ that minimizes the differences with $\mathbb{P}$ while satisfying the constraints $\mathbb{C}$:

$$\begin{aligned} &\arg \min_{\mathbb{P}'} \text{diff}(\mathbb{P}', \mathbb{P}) \\ &\text{subject to } \mathbb{P}' \text{ satisfies } \mathbb{C} \end{aligned}$$

The difference between two clusterings $\mathbb{P}$ and $\mathbb{P}'$, $\text{diff}(\mathbb{P}, \mathbb{P}')$, is measured by the number of instances that have different assignments in $\mathbb{P}$ and $\mathbb{P}'$: formally $\text{diff}(\mathbb{P}, \mathbb{P}') = \sum_{i \in [1, N]} \mathbb{1}(\mathbb{P}[i] \neq \mathbb{P}'[i])$, where $\mathbb{1}(\cdot)$ is the indicator function. Let us notice that since $\mathbb{P}$ is given and $\mathbb{P}'$ is built from it, this is not the classical problem of finding a matching between the clusters of both partitions.

The work proposed in [52] considers a particular case of this general setting in which instances are described by F numeric features and the constraints are either put on the clusters along some dimensions (for example reducing the diameter of the clusters or increasing the split between clusters), or put on the structure of the partition (merging clusters, splitting clusters, ...).

An ILP model has been developed. This model uses both Boolean variables and integer variables. For each instance $i \in [1, N]$, the following variables are defined:

- an integer variable $G_i \in [1, K]$ where $G_i = k$ means the instance is assigned to cluster C_k
- a Boolean variable z_i that is 1 (True) when $\mathbb{P}$ and $\mathbb{P}'$ disagree on i

- K Boolean variables C_{ik} , with $k \in [1, K]$, each C_{ik} determines the membership of instance i for cluster C_k .

This work focuses on feature-based diameter constraint (Subsection 3.2) defined as follows: let $\mathbb{D}'$ a $K \times F$ matrix, specifying the expected diameter of the k -th cluster along feature f , the feature-based diameter constraint is then

$$\forall k \in [1, K], \forall f \in [1, F], \text{diam}_f(C_k) \leq \mathbb{D}'[k, f]$$

where the diameter of a cluster C_k along the feature f is defined by $\text{diam}_f(C_k) = \max_{o_i, o_j \in C_k} |f(o_i) - f(o_j)|$ and $f(i)$ denotes the value taken by instance i on feature f .

The direct implementation of the feature-based diameter constraint as stated in the definition is not efficient since it involves K constraints and each constraint requires to use N^2 variables. In order to have a more efficient encoding, a 3-dimensional array $\mathfrak{d}$ is precomputed where $\mathfrak{d}[f, i, j]$ is the distance between the i -th and the j -th instances with respect to the feature f . Data is defined by $\mathbb{X}$, where $\mathbb{X}[i, f]$ gives the value of the data instance i on feature f . For each feature f , the minimum and maximum values of f in the complete set of instances, denoted respectively by $M_l[f]$ and $M_u[f]$ are also precomputed:

$$M_l[f] = \min_{i \in [1, N]} \mathbb{X}[i, f], \quad M_u[f] = \max_{i \in [1, N]} \mathbb{X}[i, f]$$

Moreover, the following decision variables are introduced:

- $L \in \mathbb{R}^{K \times F}$, where L_{kf} is the minimal value of cluster C_k along feature f
- $H \in \mathbb{R}^{K \times F}$, where H_{kf} is the maximal value of cluster C_k along feature f

The problem is formulated as follows. In this formulation, $\mathbb{P}[i]$ gives the number of cluster the instance o_i belongs to in the partition $\mathbb{P}$.

$$\begin{aligned} & \arg \min_{z, G, C, L, H} \sum_{i \in [1, N]} z_i \\ & \text{subject to} \\ & \quad \forall k \in [1, K], \forall i \in [1, N], C_{ik} = \mathbb{1}(G_i = k) \quad (\text{i}) \\ & \quad \forall i \in [1, N], z_i = \mathbb{1}(G_i \neq \mathbb{P}[i]) \quad (\text{ii}) \\ & \quad \forall k \in [1, K], \forall f \in [1, F], \quad (\text{iii}) \\ & \quad L_{kf} = \min_{i \in [1, N]} \{C_{ik}(\mathbb{X}[i, f] - M_u[f])\} + M_u[f] \\ & \quad H_{kf} = \max_{i \in [1, N]} \{C_{ik}(\mathbb{X}[i, f] - M_l[f])\} + M_l[f] \\ & \quad H_{kf} - L_{kf} \leq \mathbb{D}'[k, f] \end{aligned} \tag{1}$$

In this formulation, (i) links together Boolean and integer assignment variables, (ii) connects disagreement information and (iii) sets the diameter for each cluster C_k along each feature f . Other constraints are also encoded, as for instance

- the cardinality cluster constraint encoded by:

$$l_k \leq \sum_{i \in [1, N]} C_{ik} \leq u_k$$

- keep a cluster constraint: Let k be the number of the cluster to be kept.

$$\sum_i C_{ik} \geq 1$$

- delete cluster constraint: Let k the number of the cluster to be deleted.

$$\sum_i C_{i,k} = 0$$

- balanced binary features: This constraint sets a lower bound p_l and an upper bound p_u on the count of 1 (True) in binary features $f \in [1, F]$ for each cluster: $\forall k \in [1, K]$

$$p_l \sum_{i \in [1, N]} C_{ik} \leq \sum_{i \in [1, N]} C_{ik} \mathbb{X}[i, f] \leq p_u \sum_{i \in [1, N]} C_{ik}$$

The model (1) therefore uses N integer variables (G) and $2KF$ real variables (L, H), and $N + NK$ binary variables (z, C). The number of constraints in the ILP model is $N(K + 1) + 3KF$ which is linear in N . Encoding the other user constraints requires for instance $2K$ linear constraints for a cardinality constraint, or $2KF$ constraints for a balanced binary features constraint.

5.3.2. Partition building from a soft assignment under constraints

[62] introduces the notion of Cluster Fractional Allocation Matrix $S \in \mathbb{R}^{N \times K}$, where S_{ik} gives a score for point i to cluster k . Such a matrix can be either given by a deep/probability-based clustering (in this case S is called a soft-assignment matrix) or computed from a given partition (for instance, $S_{ik} = ||x_i - \mu_k||$, if the clusters are spherical, normalized if needed). The aim is to find a partition $\mathbb{P}$ satisfying the constraints, which is as close as possible to S . The partition $\mathbb{P}$ is represented by a Boolean matrix $C \in \{0, 1\}^{N \times K}$, where $C_{ik} = 1$ means instance o_i is assigned to cluster C_k .

The problem is formulated as

$$\begin{aligned} & \arg \min_C \sum_{i \in [1, N]} \sum_{k \in [1, K]} S_{ik} C_{ik} \\ & \text{subject to } C \end{aligned} \quad (2)$$

This problem is formulated as an ILP problem and here are some examples of constraints with their ILP formulation:

- A triplet constraint $ap | n$ is formulated by:

$$\forall k \in [1, K], C_{pk} \geq C_{ak} + C_{nk} - 1 \text{ and } C_{ak} \geq C_{pk} + C_{nk} - 1$$

This formulation yields $C_{pk} = 1$ if $C_{ak} = C_{nk} = 1$ and $C_{ak} = 1$ if $C_{pk} = C_{nk} = 1$.

- Cluster size constraint. Each cluster must contain at least α and at most β instances can be expressed by:

$$\forall k \in [1, K], \alpha \leq \sum_{i \in [1, N]} C_{ik} \leq \beta$$

- Property-cardinality constraint. Let $p(i)$ be 1 when an instance i satisfies the property p and 0 otherwise. The fact that each cluster must have at least α and at most β instances satisfying p can be enforced by:

$$\forall k \in [1, K], \alpha \leq \sum_{i \in [1, N]} p(i) C_{ik} \leq \beta$$

- Neighborhood constraint. A neighborhood constraint requires that each instance i must be in the same cluster with at least a ratio α of its neighborhood $\mathcal{N}_i$. Let $N_i(j) = 1$ if instance j is in the neighborhood of i and 0 otherwise. This constraint is expressed by:

$$\forall i \in [1, N], \forall k \in [1, K], \sum_{j \in [1, N]} N_i(j) C_{jk} \geq \alpha \left(\sum_{j \in [1, N]} N_i(j) \right) C_{ik}$$

- Span limited constraint limiting the number of possible clusters for the instances having a specific property. Let $p(i)$ be 1 when the instance i satisfies p and 0 otherwise and let t_k be a Boolean variable for each cluster k that is true if and only if cluster k contains some points satisfying p . t_k is binded by:

$$t_k \leq \sum_i p(i) C_{ik} \text{ and } \forall i \in [1, N], t_k \geq p(i) C_{ik}$$

Bounds $[\alpha, \beta]$ on the number of clusters containing instances having the property p are given by: $\alpha \leq \sum_{k \in [1, K]} t_k \leq \beta$.

This framework has been applied to ensemble clustering under constraints in [45]: given a set of partitions, a consensus partition is built, a soft assignment matrix S is computed from this partition and the best partition according to S and satisfying the constraints is computed.

6. Conceptual clustering by assignment of instances

Declarative methods for conceptual clustering by assignment of instances to clusters are based on CP or on SAT. The methods propose a very general constraint-based language for Data Mining problem (e.g. pattern mining), in which conceptual clustering problems can be expressed. The common point of these approaches is that they use binary variables to encode the constraints, either using SAT or CP.

Conceptual clustering is formulated as finding a partition $\mathbb{P}$ of K clusters, where each cluster C_k is associated to a closed pattern P_k , therefore $\mathbb{P}$ is written $\mathbb{P} = \{(C_1, P_1), \dots, (C_K, P_K)\}$, such that the clusters do not overlap and cover all the objects. The constraints are stated as below.

$$\begin{aligned}
& \text{coverage}(C_k, P_k) \wedge \text{closed}(C_k, P_k) && \forall k \in [1, K], \text{ conceptual constraints} \\
& \bigcup_{k=1}^K C_k = \mathcal{O} && \text{all instances covered constraint} \\
& \text{notEmpty}(C_k) && \forall k \in [1, K], \text{ not empty cluster constraints} \\
& C_k \cap C_{k'} = \emptyset && \forall 1 \leq k < k' \leq K, \text{ non overlap constraints}
\end{aligned} \tag{3}$$

Let N be the number of instances, B the number of properties and K the number of clusters. The dataset is represented by a matrix $\mathbb{B}$ of size $N \times B$, with $\mathbb{B}[i, p] = 1$ iff instance i has property p . In order to define a partition, both CP and SAT methods use binary variables as follows.

- To define the assignment of the instances to the clusters, a matrix C of $N \times K$ Boolean variables is defined, where $C_{ik} = 1$ (or *True*) iff instance o_i is assigned to cluster C_k .
- In the same manner for properties, a matrix P of $B \times K$ Boolean variables is defined, where $P_{pk} = 1$ (or *True*) iff property p is in pattern P_k .

In this way, P_k is the set of properties p where $P_{pk} = 1$ and C_k is the set of instances o_i where $C_{ik} = 1$. Therefore, to find a set of K clusters, the number of Boolean variables are $K(N + B)$.

6.1. Encoding constraints in CP

The first work that built the bridge between Data Mining and Constraint Programming was [66], where the itemset mining problem was modeled in a CP framework. In this work, properties of an itemset such as frequency or closedness are formulated and modeled as a CSP in CP. Mining frequent or closed itemsets becomes solving the CSP. Another CP based method was proposed in [50] for modeling n-ary pattern mining, which are patterns satisfying specific conditions, such as exception rules.

A method for modeling conceptual clustering problems has been presented in [43]. This work extends [66] to model k-pattern set mining problem. General constraints and operations are defined to express properties of individual patterns or between several patterns.

Using the Boolean variables C and P , the constraints are encoded as follows. For each couple (C_k, P_k) , the constraint $\text{coverage}(C_k, P_k)$ is expressed by N CP reified constraints, one for each instance $i \in [1, N]$:

$$C_{ik} \leftrightarrow \sum_{p \in [1, B]} P_{pk}(1 - \mathbb{B}[i, p]) = 0$$

The constraint $\text{closed}(C_k, P_k)$ is expressed by B reified constraints, one for each property $p \in [1, B]$:

$$P_{pk} \leftrightarrow \sum_{i \in [1, N]} C_{ik}(1 - \mathbb{B}[i, p]) = 0$$

For all values $k, k' \in [1, K]$ with $k < k'$, the non overlap constraint $C_k \cap C_{k'} = \emptyset$ is encoded by one constraint

$$\sum_{i \in [1, N]} \mathbb{1}(C_{ik} + C_{ik'} = 2) = 0$$

This constraint enforces that the number of objects i for which both C_{ik} and $C_{ik'}$ are 1 is zero. The constraint that all the instances are covered is expressed by N CP-constraints, one for each $i \in [1, N]$

$$\sum_{k=1}^K C_{ik} \geq 1$$

which means each object i is covered by at least one cluster.

Therefore, the conceptual clustering is encoded by $K(N + B)$ Boolean variables and $K(N + B + (K - 1)/2) + N$ CP-constraints.

6.2. Encoding constraints in SAT

A SAT-based framework for conceptual clustering has been developed in [56]. In this paper, a constraint-based language is proposed, which provides a set of built-in functions and a set of constraints on objects or on the clusters. Using this language, one can express by a query the properties that the output clustering must satisfy.

Exploiting Boolean variables, constraints are expressed by CNF formulas. The constraint requesting that all the instances are covered, which means each instance $i \in [1, N]$ belongs to at least one cluster, is encoded by

$$\bigwedge_{i \in [1, N]} \left(\bigvee_{k \in [1, K]} C_{ik} \right)$$

This formula is encoded by N SAT clauses, each of them contains K literals. The non overlap constraint stating that each instance $i \in [1, N]$ belongs to at most one cluster is encoded by

$$\bigwedge_{i \in [1, N]} \left(\bigwedge_{k, k' \in [1, K], k < k'} (\neg C_{ik} \vee \neg C_{ik'}) \right)$$

This formula is therefore encoded by $NK(K-1)/2$ clauses, each one is of 2 literals, must-link/cannot-link constraint on two instances i, j can be also encoded as follows:

$$\begin{aligned} \text{mustLink}(i, j) &\equiv \bigwedge_{k \in [1, K]} (\neg C_{ik} \vee C_{jk}) \wedge (C_{ik} \vee \neg C_{jk}) \\ \text{cannotLink}(i, j) &\equiv \bigwedge_{k \in [1, K]} (\neg C_{ik} \vee \neg C_{jk}) \end{aligned}$$

The formulation for `cannotLink` differs from the formulation given in [56], which is valid only for $K = 2$. The clustering problem is therefore encoded by CNF which is a set of SAT clauses. SAT solvers find one assignment of the variables in order to satisfy all the clauses. In order to find all the assignments, each time a solution s is found, the clause $\neg s$ is added to the CNF and SAT solver is restarted to find another solution. This process is repeated until failure, i.e. no more solution exists.

6.3. Descriptive clustering

Descriptive clustering [29] aims at finding a clustering that is good both in terms of cluster homogeneity based on a distance measure, and in term of clusters description using the properties of the instances, as described in Subsection 3.3. The instances are described by a set $\mathcal{X}$ of features and a set $\mathcal{B}$ of Boolean descriptors. The two sets $\mathcal{X}$ and $\mathcal{B}$ may be non-disjoint. The pairwise distance metric d is computed on $\mathcal{X}$, i.e. $d : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$. The aim is to build a partition $\mathbb{P}$ based on d such that each cluster has a description which is a non empty subset of $\mathcal{B}$.

Two optimization criteria must therefore be considered, one criterion to characterize the clusters homogeneity based on the distance d (feature-focused or compactness), the other criterion to characterize the quality of the cluster description (descriptor-focused or descriptiveness). Descriptive clustering aims at finding a clustering that optimizes both criteria, i.e. each cluster is compact, according to a feature-focused objective f , and descriptive, using a descriptor-focused objective g . [29] empirically shows that a criterion cannot be optimized without worsening the other. This work therefore considers descriptive clustering as a bi-objective optimization problem, with two objectives f and g , and computes the Pareto optimal solutions as a trade-off between these objectives.

Simultaneously finding compact and descriptive clusters. Let f be a compactness objective to be minimized and g be a descriptiveness to be maximized. A solution $\mathbb{P}$ dominates another solution $\mathbb{P}'$ if $f(\mathbb{P}) \leq f(\mathbb{P}')$, $g(\mathbb{P}) \geq g(\mathbb{P}')$, and either $f(\mathbb{P}) < f(\mathbb{P}')$ or $g(\mathbb{P}) > g(\mathbb{P}')$. A solution is Pareto optimal if it is not dominated by any other solution. Let C be the constraints stating the clustering and description requirements (see the following paragraph). Algorithm 1 presents a general iterative scheme to find a complete and minimal set of Pareto optimal solutions using C as sub-problems. This algorithm follows the scheme proposed in [25] where a proof of completeness is given.

Algorithm 1: Compute complete Pareto front.

Input: Features $\mathcal{X}$, descriptors $\mathcal{B}$, clustering/description constraints C

Output: A complete Pareto front $\mathcal{P}$.

```

1  $\mathcal{P} \leftarrow \emptyset$ ;
2  $\mathbb{P}_1^f \leftarrow \arg \min f$  subject to  $C$ ;
3  $i \leftarrow 1$ ;
4 while  $\mathbb{P}_i^f \neq \text{NULL}$  do
5    $\mathbb{P}_i^g \leftarrow \arg \max g$  subject to  $C \cup \{f \leq f(\mathbb{P}_i^f)\}$ ;
6    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\mathbb{P}_i^g\}$ ;
7    $i \leftarrow i + 1$ ;
8    $\mathbb{P}_i^f \leftarrow \arg \min f$  subject to  $C \cup \{g > g(\mathbb{P}_{i-1}^g)\}$ ;
9 return  $\mathcal{P}$ ;
```

Modeling description requirements. Two methods have been proposed based on ILP for one and on CP for the other to model clustering and description requirements. Both methods use the Boolean variables C and P to represent the clusters and the cluster descriptions. Using these variables, the constraints are encoded as below.

- Non-empty description:

$$\sum_{p \in [1, B]} P_{pk} \geq 1, \quad \forall k \in [1, K]$$

- Description constraints: An instance in a cluster C_k must satisfy most of its descriptions P_k (up to $\alpha \geq 0$ exceptions). $N \times K$ constraints are necessary, one for each $i \in [1, N]$ and for each $k \in [1, K]$:

$$C_{ik} = 1 \rightarrow \sum_{p \in [1, B]} P_{pk} (1 - \mathbb{B}[i, p]) \leq \alpha$$

- **Almost-closed constraints:** A descriptor is included in a cluster description P_k if and only if most of the instances in the cluster C_k (up to $\beta \geq 0$ exceptions) satisfy it. $K \times B$ constraints are used, one for each $p \in [1, B]$ and for each $k \in [1, K]$:

$$P_{pk} = 1 \leftrightarrow \sum_{i \in [1, N]} C_{ik} (1 - \mathbb{B}[i, p]) \leq \beta$$

Modeling partition and compactness criterion. Using the variables C_{ik} partition constraints are expressed as in Subsection 6.1. Moreover, to take advantage of constraints on integer variables, CP-based method also uses integer variables $G_i \in [1, K]$ as in Subsection 5.2, where $G_i = k$ means that instance o_i is assigned to cluster C_k . The following constraints link the two types of variables

$$\forall i \in [1, N], \forall k \in [1, K], C_{ik} = 1 \leftrightarrow G_i = k$$

The use of these integer variables enables the integration of global optimization constraints, which were developed for the compactness criterion, such as minimizing the maximal cluster diameter [25] or minimizing the sum of within-cluster distances [23]. They are also exploited to express the descriptiveness criterion as stated below.

Descriptiveness criteria. Three descriptor-focused/descriptiveness criteria have been introduced, depending on the situation whether the properties are sparse/noisy or not.

- **Maximize minimal complete tag agreement (MMCTA):** this is the strongest objective when the properties are well populated and have little noise. It requires that in each cluster C_k each instance shares the properties P_k with *all* the other instances in the cluster. Therefore the description constraint is stated with $\alpha = 0$ and the almost-closed constraint with $\beta = 0$. The aim is to maximize the smallest description for a cluster:

$$\text{maximize } \min_{(C, P) \in \mathbb{P}} \#P$$

- **Minimize tag disagreement (MTD):** when the properties are sparse and/or have noise, disagreements are allowed in the description and the almost-closed constraints ($\alpha \geq 0, \beta \geq 0$). The objective is to minimize these disagreements

$$\text{minimize } \alpha + \beta$$

- **Maximize minimal neighborhood agreement (MMNA):** this objective requires each instance shares at least q properties with another instance in the same cluster and aims at maximizing q :

$$\text{maximize } \min_{o, o' \in \mathcal{O} | \mathbb{P}[o] \neq \mathbb{P}[o']} \#\{i \in B \mid \mathbb{B}[o, i] = \mathbb{B}[o', i] = 1\}$$

The Max-Min Complete Tag Agreement (MMCTA) is to maximize Q , with Q being a variable representing the size of the shortest description for a cluster:

$$Q = \min_{k \in [1, K]} \left(\sum_{p \in [1, B]} P_{pk} \right)$$

The Max-Min Neighborhood Agreement (MMNA) is a weaker form, where in each cluster, every instance must share at least Q tags with another instance in the cluster. The objective function is to maximize Q , where

$$\forall i < j \in [1, N], G_i = G_j \rightarrow \sum_{p \in [1, B]} \mathbb{B}[i, p] \mathbb{B}[j, p] \geq Q \quad (4)$$

which can be encoded by $N \times (N - 1)/2$ reified CP constraints. For these two objective functions, it is observed that if Q is the objective variable with the domain defined by a lower bound $\underline{Q}$ and an upper bound $\overline{Q}$, any two instances sharing less than $\underline{Q}$ tags should not be in the same cluster. Instead of using reified constraints, one global optimization constraint has been therefore developed to enforce the relation (4). This constraint revises the upper bound $\overline{Q}$ using the variables G_i that have been assigned and also propagates $G_i \neq G_j$ for all pairs of instances o_i, o_j that share less than $\underline{Q}$ tags.

7. Constrained clustering by cluster selection

While clustering by assignment methods consider finding a partition by creating the clusters (by assigning instances to clusters), clustering by selection methods take another point of view of the clustering problem: instead of creating the clusters, they consider selecting K clusters among all the possible ones to form the partition $\mathbb{P}$.

Let C be the set of all the possible clusters. Let $\mathbb{A} \in \{0, 1\}^{N \times |C|}$ be a matrix such that $\mathbb{A}[i, c] = 1$ iff the cluster $c \in C$ contains instance $o_i \in \mathcal{O}$. The problem can be formulated by the following Integer Linear Program:

$$\begin{aligned}
& \text{minimise} && \sum_{c \in C} v[c] X_c, \\
& \text{subject to} && \sum_{c \in C} \mathbb{A}[i, c] X_c = 1, \quad \forall i \in [1, N] \\
& && \sum_{c \in C} X_c = K, \\
& && X_c \in \{0, 1\} \quad \forall c \in C
\end{aligned} \tag{5}$$

In this formulation, one binary variable X_c is defined for each cluster $c \in C$, such that $X_c = 1$ iff the cluster c is selected for the partition $\mathbb{P}$. The value $v[c]$ is the quality value computed beforehand for the cluster c , according to the aimed clustering criterion. The cluster quality $v[c]$ can be for instance the WCSS of the cluster c . The objective function represents therefore finding a partition minimizing the WCSS criterion. The first constraint enforces that each point must be covered by exactly one cluster. The second one enforces that there are K selected clusters in the final partition. However, in this formulation, since the number of possible clusters is 2^N for N points, the number of variables is 2^N which is too large for the program to be solved.

Clustering by cluster selection methods needs therefore a set of cluster candidates C that can be handled. The cluster candidates can be generated once at the beginning or on the fly. In conceptual clustering, the considered clusters can be either frequent patterns or closed patterns. They can be therefore generated using dedicated algorithms. Methods for conceptual constrained clustering will be presented in Subsection 7.1. In distance based clustering, cluster candidates can be generated on the fly. Methods in this setting will be presented in Subsection 7.2.

7.1. Conceptual constrained clustering by cluster selection

In conceptual clustering, the clusters are required to correspond to frequent patterns or closed patterns. This requirement allows to restrict and to generate beforehand the set of candidate clusters C' . A two-step method based on selection of clusters proceeds by:

1. Generating a set C' of candidates clusters.
2. Selecting among all the candidates the clusters to form a partition, based on an objective function and subject to constraints on the instances, on the clusters or on the clustering itself. The formulation (5) is therefore defined with the set C' of candidates clusters.

Following this process, [58] and [63] have developed ILP-based frameworks that can integrate different kinds of user-constraints for constrained conceptual clustering. The restricted subset C' of clusters consists of the set of all the frequent patterns [58] or the set of all the closed patterns [63].

It can be observed that instance-level constraints (such as must-link and cannot-link) and cluster-level constraints (such as for instance maximal or minimal cluster cardinality) can be verified for each cluster candidate. Therefore, both [58] and [63] use the same principle, where instance-level and cluster-level constraints are enforced to remove cluster candidates that do not satisfy the constraints.

Moreover, from the point of view of cluster selection, constraints on the clustering itself can be expressed. Below are examples of these types of constraints together with their ILP encoding [58].

- At least α clusters in a given list L ($L \subseteq C'$) of clusters are selected. This constraint can be encoded by the linear constraint:

$$\sum_{c \in L} X_c \geq \alpha$$

The constraint that at most β clusters are selected can be expressed in a similar way.

- The selection (or not) of some clusters depends on the selection (or not) of some other clusters. For instance if a cluster c_1 is selected and c_2 is not, then a cluster c_3 must be selected. This constraint can be encoded by

$$X_{c_3} - X_{c_1} - (1 - X_{c_2}) \geq -1, \text{ i.e. } X_{c_3} - X_{c_1} + X_{c_2} \geq 0$$

which is only violated for $X_{c_1} = 1$, $X_{c_2} = 0$ and $X_{c_3} = 0$.

- In the classic setting of clustering, each data instance must be in a cluster. This restriction can be relaxed by requiring that at least ϕ instances and at most ψ instances are covered. When $\psi = \phi = N$, all the instances must be covered by a cluster. In this case the constraints put in formulation (5):

$$\sum_{c \in C'} \mathbb{A}[i, c] X_c = 1, \quad \forall i \in [1, N]$$

which requires each instance to be covered by one and exactly one cluster is replaced by the following constraints, introducing new variables Y_i for all instances $o_i \in \mathcal{O}$, where $Y_i = 1$ iff o_i is in one selected cluster and 0 otherwise:

$$\begin{aligned} \sum_{c \in C'} \mathbb{A}[i, c] X_c &= Y_i, & \forall i \in [1, N] \\ \phi &\leq \sum_{i \in [1, N]} Y_i \leq \psi \\ Y_i &\in \{0, 1\}, & \forall i \in [1, N] \end{aligned}$$

- In a partition, each data instance belongs to only one cluster. This condition can also be relaxed by stating that each instance must belong to at least α and at most β clusters:

$$\alpha \leq \sum_{c \in C'} \mathbb{A}[i, c] X_c \leq \beta, \quad \forall i \in [1, N]$$

When $\alpha = \beta = 1$, this means each instance is covered by exactly one cluster.

The objective function can also be chosen among several options [58]:

- *maxMeanQuality*: the mean quality of the selected clusters is optimized. The objective function is

$$\text{maximize } \frac{1}{K} \sum_{c \in C'} (v_{\max} - v[c]) X_c$$

where $v[c]$ is the quality of the cluster c and $v_{\max}$ is a new variable with the constraint $v_{\max} = \max_{c \in C'} (v[c] X_c)$.

- *maxMinQuality*: the minimum quality of the selected clusters is optimized. The objective function is maximize $d_{\min}$, where $d_{\min}$ is a new variable linked with the constraints

$$d_{\min} \leq v_{\max} - v[c] X_c, \quad \forall c \in C'$$

- *maxMedianQuality*: the median quality of the selected clusters is optimized.

Constraints on clustering also enable the modeling of soft clustering (at most some instances can be uncovered or small overlaps are allowed), co-clustering (clustering that covers both the set of instances and the set of properties, without any overlap on instances or on properties) and soft co-clustering. For instance, the following program models the co-clustering problem: finding K clusters covering both $\mathcal{O}$ and $\mathcal{B}$ without overlap, with K varying from $K_{\min}$ to $K_{\max}$ [63]. Here $\mathbb{W} \in \{0, 1\}^{|\mathcal{B}| \times |C'|}$ is a constant matrix, where $\mathbb{W}[p, c] = 1$ iff instances of the cluster c have property p .

$$\begin{aligned} &\text{optimize } \sum_{c \in C'} v[c] X_c \\ &\text{subject to } \sum_{c \in C'} \mathbb{A}[i, c] X_c = 1 & \forall i \in [1, N] & \quad \text{(i)} \\ & \sum_{c \in C'} X_c = K & & \quad \text{(ii)} \\ & K_{\min} \leq K \leq K_{\max} \\ & \sum_{c \in C'} \mathbb{W}[p, c] X_c = 1 & \forall p \in [1, B] & \quad \text{(iii)} \\ & k \in \mathbb{N}, X_c \in \{0, 1\}, & \forall c \in C' \end{aligned}$$

In this formulation, (i) states that each instance is covered by exactly one cluster, (ii) that there are K clusters which can range from $K_{\min}$ to $K_{\max}$, and (iii) that each property is covered by exactly one cluster.

The same two-step constrained conceptual clustering using CP is also developed [16]. In the second step of this method, it is observed that since each object is covered by exactly one cluster, the cluster selection step can be seen as an instance of the exact cover problem. A global constraint

$$\text{exactCover}_{Q, C', q}(\text{selected}, \text{MinQ}, \text{MaxQ})$$

is therefore developed, which enforces that

- the *selected* variables assigned to *true* correspond to an exact cover of $(\mathcal{O}, C')$,
- *MinQ* and *MaxQ* variables are assigned to the minimum and maximum quality associated with the selected subsets.

7.2. Distance-based constrained clustering by cluster selection

For distance-based clustering, methods based on cluster selection, similar to the ILP formulation in (5) have been developed in ILP and use column generation to generate new cluster candidates (columns). Let us notice that in the ILP model (5), each variable

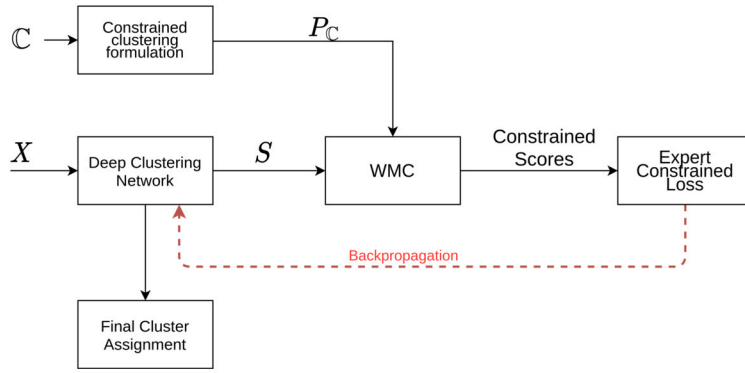


Fig. 2. Knowledge integration in deep clustering.

of the model corresponds to a candidate cluster. The number of variables can be exponential to the number of data instances. Indeed for N data points, the number of variables, which is the number of possible clusters, is 2^N . An ILP-based approach with column generation for unconstrained WCSS clustering was introduced by [31] and improved by [2]. In this method, a restricted master problem is defined, which is the initial problem restricted to a smaller set of variables $C' \subseteq C$. Column generation iterates between solving the restricted master problem and adding one or multiple variables (columns). A variable is added to the master problem if it can improve the objective function. If no such variables can be found, the optimal solution of the restricted master problem is also an optimal solution of the full master problem. To decide whether a variable can improve the objective function, the dual of the master problem is solved. A variable that improves the objective function of the master problem corresponds to a variable with a negative reduced cost in the dual. Among all the variables with negative reduced cost, the smallest one is usually searched for, which yields a minimization subproblem.

In [9], it is observed that many user-constraints can be evaluated on each cluster individually, therefore the user-constraints are not part of the master problem, but only have to be considered when solving the subproblems. The constraints are therefore enforced when solving the subproblem by removing all the variables corresponding to the clusters that do not satisfy the constraints. Identifying variables (cluster candidates) was then represented by a constrained optimization problem, and a constrained branch-and-bound algorithm was developed to solve it. This method can integrate must-link and cannot-link constraint, and any anti-monotonic constraints. A constraint is anti-monotone iff its satisfaction on a cluster C implies its satisfaction on all sub-clusters $C' \subseteq C$. For instance a maximum cluster size constraint is anti-monotonic while a minimum cluster size constraint is not. Anti-monotonic constraints can be integrated since the bounds used in the branch-and-bound algorithm remain valid under such constraints.

8. Constrained clustering using deep learning

Deep clustering methods allow to learn at the same time a new representation space (more suited to the clustering task) [78,42,12,75] and a clustering. From a general point of view, a deep neural network is organized with neurons, which are interconnected and structured in layers. Each connection between two neurons of different layers has a weight. An activation function is defined for each neuron that computes an output based on the weighted input values. The output of the final layer is the model output. Training the model consists in determining the weights and this is achieved by minimizing a loss function, which defines the difference between the model output and the expected output. Since clustering is an unsupervised task, self training models are usually used: an initial classifier is created to label the data, for instance by applying a clustering method and assigning to each cluster a number, which becomes the label of its instances, and then the labels that are predicted with high probabilities are enforced to improve the classifier.

In order to integrate user constraints, [81] proposes to define a loss for each type of constraints depending on the meaning of the constraints. The final loss is therefore a combination of several losses.

Following the principle of declarative approaches, which allow to express in a same formalism different kinds of constraints, [61] proposes to define a general expert loss, inspired by the work of [79] for semi-supervised classification. This loss can be combined with a classic deep clustering loss to train the model. A general overview of the framework is presented in Fig. 2. In this framework, constraints are expressed in propositional logic, thus encapsulating a large diversity of constraints. The expert loss is then computed through a Weighted Model Counting (WMC) problem [69]. Thanks to this loss, constraints can be integrated in any deep clustering method under the condition that this method computes a soft assignment of points to clusters (i.e. assigns to each point and to each cluster a probability for this point to belong to the cluster). The expert loss can be added to the other losses and back-propagated inside the deep clustering framework. When a stopping condition is reached (for instance the change of the loss is under a given threshold), a final assignment is computed by assigning each point to the cluster having the largest probabilities. The integration of constraints into two different deep clustering frameworks such as IDEC [42] and SCAN [75] has been presented in [61]. The computation of the expert loss is presented more in details below.

Expert loss based on a constraint satisfaction score. Let S be a soft assignment obtained by a deep clustering method: S_{ij} gives the probability of point i to belong to cluster j . Let C be a constraint set. A partition is a hard assignment of points to clusters and given S , the appropriateness of a partition $\mathbb{P}$ to S can be assessed by:

$$Score(\mathbb{P}, S) = \prod_{i \in [1, N]} S_{i, \mathbb{P}[i]} \quad (6)$$

where $\mathbb{P}[i]$ denotes the cluster that o_i is assigned to in partition $\mathbb{P}$. This score is all the higher that the partition $\mathbb{P}$ assigns points to its most probable cluster. In presence of constraints the partition with the highest score may not satisfy constraints.

This score between a partition and a constraint is not used to find the best partition satisfying the constraints but to assess the adequacy of a soft assignment S to the constraints C . $Score(C, S)$ measures how S fits with the partitions satisfying the constraints C . It is defined by:

$$Score(C, S) = \sum_{\mathbb{P} \text{ satisfying } C} Score(\mathbb{P}, S) = \sum_{\mathbb{P} \text{ satisfying } C} \prod_{i \in [1, N]} S_{i, \mathbb{P}[i]}$$

The expert loss is defined on $Score(C, S)$. It could have been defined as $-\log(Score(C, S))$ but to have a greater impact when back-propagating the loss and to reduce the complexity, C is decomposed as if the constraints were independent:

$$L_{expert} = \sum_{C \in \mathcal{C}} -\log(Score(C, S))$$

Expert loss computation. The computation of the expert loss requires computing the score over all the partitions satisfying a constraint C . However enumerating all these partitions is expensive. Instead it has been shown in [61] that the computation can be converted into a Weighted Model Counting problem. Let $\mathbb{F}$ be a propositional formula written in Conjunctive Normal Form with variables $V_1, \dots, V_l$ such that each variable V_i is associated to a weight w_i . The weight of an assignment $\mathbf{v}$ of Boolean values to variables is computed as $\prod_{i=1}^l \gamma_i$ with $\gamma_i = w_i$ when V_i is set to *True* (i.e. $\mathbf{v} \models V_i$) and $\gamma_i = 1 - w_i$ when V_i is set to *False* (i.e. $\mathbf{v} \models \neg V_i$). The Weighted Model Counting (WMC) [69] of $\mathbb{F}$ is defined by:

$$WMC(\mathbb{F}, w) = \sum_{\mathbf{v} \models \mathbb{F}} \prod_{i \models V_i} w_i \prod_{i \not\models V_i} (1 - w_i)$$

which allows to compute the sum of the weights of all the models of $\mathbb{F}$. In this formula, the weights w_i and $1 - w_i$ can occur several times. Sentential Decision Diagrams (SDD) [17] allows a more efficient computation by compressing it into an arithmetic circuit.

Relying on WMC for computing $Score(C, S)$ is not straightforward, since according to (6) only the weights for variables set to *True* is used. To achieve this, [61] introduces a set of Boolean variables $C \in \{0, 1\}^{N \times K}$ where $C_{ik} = 1$ means instance o_i is assigned to cluster C_k . Enforcing partition requirements, a formula β_{ik} is defined for $i \in [1, N]$, $k \in [1, K]$, to state that o_i is put only in the cluster C_k (and not in the other clusters). It is defined as:

$$\begin{aligned} \beta_{ik} &\stackrel{def}{=} C_{ik} \wedge \bigwedge_{l \in [1, k-1]} \neg C_{il} \quad \text{if } 1 \leq k \leq K-1, \\ \beta_{iK} &\stackrel{def}{=} \bigwedge_{l \in [1, K-1]} \neg C_{il} \end{aligned}$$

The weight for each variable C_{ik} is defined by:

$$w_B(C_{ik}) = \begin{cases} \frac{S_{ik}}{1 - \sum_{l \in [1, k-1]} S_{il}} & \text{if } \sum_{l \in [1, k-1]} S_{il} < 1 \\ 1 & \text{otherwise} \end{cases}$$

It is shown that any assignment satisfying the $\bigwedge_{i \in [1, N], k \in [1, K]} \beta_{ij}$ forms a partition and that given a constraint C :

$$Score(C, S) = WMC(C \wedge \bigwedge_{i \in [1, N], k \in [1, K]} \beta_{ij}, w_B)$$

Constraint integration. Constraints can therefore be integrated as soon as they are expressed in propositional logic. The constraints that are implemented in [61] are constraints on points, that are pairwise and triplet constraints, implication constraints on the relation between points and a new constraint called span-limited constraint. They are all defined by propositional formulae and defined on the formula β_{ik} . Let us define the predicates *Together*(u, v) and *Apart*(u, v) meaning that two points are (resp. are not) in the same cluster by

$$Together(u, v) \stackrel{def}{=} \bigwedge_{k \in [1, K]} ((\beta_{uk} \wedge \beta_{vk}) \vee (\neg \beta_{uk} \wedge \neg \beta_{vk}))$$

and

$$Apart(u, v) \stackrel{def}{=} \bigwedge_{k \in [1, K]} (\neg \beta_{uk} \vee \neg \beta_{vk})$$

These formulae allow to represent must-link and cannot-link constraints. A triplet constraints (a, p, n) expressing that a is closer to p than to n is expressed by:

$$Together(a, n) \rightarrow Together(a, p)$$

More complex implication constraints can also be expressed.

The span-limited constraint expresses that a given group I of points should be dispatched in at most a given number m of clusters. This constraint can be encoded by:

$$\forall J \subseteq [1, K], |J|=m \wedge_{i \in I} \vee_{k \in J} \beta_{ik}$$

Let us notice that the use of propositional logic makes it more difficult to encode constraints requiring numeric values (cardinalities constraints, geometric constraints, etc.).

9. Conclusion

In this paper we have considered two views of clustering, namely distance-based clustering and conceptual clustering, that are usually studied by different researcher communities. We have proposed an inventory of constraints coming from both domains and presented declarative methods for integrating such constraints in a clustering process.

The declarative methods allow to integrate different types of constraints. Pure declarative methods enable finding solutions satisfying all the constraints when such a solution exists, and find a global optimal solution. These characteristics can be advantageous when the constraints are of high importance for the experts and need to be satisfied. This is no longer true when declarative methods are integrated into a more global architecture, as for instance in a deep learner.

However finding an optimal solution can be an issue for the efficiency, since the search space needs to be completely explored. Constraints can help reducing the search space but this is highly dependent on the form of the constraint, its appropriateness to the chosen declarative framework (for instance linear constraint for ILP) and its encodings in this framework. The choice of the declarative framework is important and difficult: most constraints can be represented both in ILP, in CP or in SAT but for instance ILP is more adapted when constraints are all linear with a reasonable number of decision variables and constraints whereas CP is more appropriate with constraints representing discrete relations. Moreover, in CP it is possible to guide the search for a solution by search strategies and to develop specific global constraints to reduce the search space.

Declarative methods offer an undeniable ease of designing a new constrained clustering problem, by specifying the optimization criterion and conditions on the desired solution, expressed by constraints. Descriptive clustering which combines requirements coming from distance-based and conceptual clustering can be therefore naturally expressed. Another illustration is given in conceptual clustering where a same framework can be used for soft-clustering or for co-clustering [63]. Another interest of being able to change the optimization criterion and to add new constraints is to address a bi-objective constrained clustering problem. This problem aims to find clusters that are both compact (minimizing the maximal diameter) and well separated (maximizing the split), under user constraints. In [25] it is shown that to solve this problem, the framework can be used by iteratively changing the objective function and adding constraints on the other objective value.

An interesting research direction would be to integrate preferences on constraints. This can be achieved using weighted constraints. It could be done in a deep architecture by putting different weights on the loss function associated to the constraint but without an explicit control on the constraints that will be satisfied.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

The authors are supported by the ANR project InvoluD (ANR-20-CE23-0023), the ANR project HERELLES (ANR-20-CE23-0022) and the ANR project AI.iO (ANR-20-THIA-0017).

References

- [1] Daniel Aloise, Amit Deshpande, Pierre Hansen, Preyas Popat, Np-hardness of euclidean sum-of-squares clustering, *Mach. Learn.* 75 (2) (May 2009) 245–248.
- [2] Aloise Daniel, Pierre Hansen, Leo Liberti, An improved column generation algorithm for minimum sum-of-squares clustering, *Math. Program.* 131 (1–2) (2012) 195–220.
- [3] Rakesh Agrawal, Tomasz Imielinski, Arun N. Swami, Mining association rules between sets of items in large databases, in: Peter Buneman, Sushil Jajodia (Eds.), *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, Washington, DC, USA, May 26–28, 1993, ACM Press, 1993, pp. 207–216.
- [4] Sugato Basu, Mikhail Bilenko, Arindam Banerjee, Raymond J. Mooney, Probabilistic semi-supervised clustering with constraints, in: Olivier Chapelle, Bernhard Schölkopf, Alexander Zien (Eds.), *Semi-Supervised Learning*, The MIT Press, 2006, pp. 73–102.
- [5] P. Bradley, K. Bennett, A. Demiriz, Constrained k-means clustering, Technical Report MSR-TR-2000-65, Microsoft Research, 2000.

- [6] S. Basu, A. Banerjee, R. Mooney, Active semi-supervision for pairwise constrained clustering, in: *Proceedings of the SIAM International Conference on Data Mining*, 2004, pp. 333–344.
- [7] S. Basu, M. Bilenko, R. Mooney, A probabilistic framework for semi-supervised clustering, in: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2004, pp. 59–68.
- [8] M. Bilenko, S. Basu, R. Mooney, Integrating constraints and metric learning in semi-supervised clustering, in: *Proceedings of the International Conference on Machine Learning*, 2004, pp. 11–18.
- [9] Behrouz Babaki, Tias Guns, Siegfried Nijssen, Constrained Clustering Using Column Generation, in: Helmut Simonis (Ed.), *Integration of AI and OR Techniques in Constraint Programming - 11th International Conference, CPAIOR 2014, Cork, Ireland, May 19-23, 2014. Proceedings*, in: *Lecture Notes in Computer Science*, vol. 8451, Springer, 2014, pp. 438–454.
- [10] Armin Biere, Marijn Heule, Hans van Maaren, Toby Walsh (Eds.), *Handbook of Satisfiability*, second edition, *Frontiers in Artificial Intelligence and Applications*, vol. 336, IOS Press, 2021.
- [11] Michael J. Brusco, A repetitive branch-and-bound procedure for minimum within-cluster sums of squares partitioning, *Psychometrika* 71 (2) (2006) 347–363.
- [12] Mathilde Caron, Piotr Bojanowski, Armand Joulin, Matthijs Douze, Deep clustering for unsupervised learning of visual features, in: Vittorio Ferrari, Martial Hebert, Cristian Sminchisescu, Yair Weiss (Eds.), *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XIV*, in: *Lecture Notes in Computer Science*, vol. 11218, Springer, 2018, pp. 139–156.
- [13] Sumit Chopra, Raia Hadsell, Yann LeCun, Learning a similarity metric discriminatively, with application to face verification, in: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2005)*, 20-26 June 2005, San Diego, CA, USA, IEEE Computer Society, 2005, pp. 539–546.
- [14] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Sergei Vassilvitskii, Fair clustering through fairlets, in: *Advances in Neural Information Processing Systems*, 2017, pp. 5029–5037.
- [15] Stephen A. Cook, The complexity of theorem-proving procedures, in: Michael A. Harrison, Ranan B. Banerji, Jeffrey D. Ullman (Eds.), *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, May 3-5, 1971, Shaker Heights, Ohio, USA, ACM, 1971, pp. 151–158.
- [16] Maxime Chabert, Christine Solnon, Constraint programming for multi-criteria conceptual clustering, in: *Principles and Practice of Constraint Programming - 23rd International Conference, CP 2017, Proceedings*, 2017, pp. 460–476.
- [17] Adnan Darwiche, Sdd: a new canonical representation of propositional knowledge bases, in: *IJCAI*, 2011.
- [18] Ian Davidson, Clustering with constraints, in: Ling Liu, M. Tamer Özsu (Eds.), *Encyclopedia of Database Systems*, second edition, Springer, 2018.
- [19] A. Demiriz, K. Bennett, P. Bradley, Using assignment constraints to avoid empty clusters in k-means clustering, in: S. Basu, I. Davidson, K. Wagstaff (Eds.), *Constrained Clustering: Advances in Algorithms, Theory, and Applications*, 1 edition, Chapman & Hall/CRC, 2008, pp. 201–220, chapter 9.
- [20] Ayhan Demiriz, Kristin P. Bennett, Paul S. Bradley, Using assignment constraints to avoid empty clusters in k-means clustering, in: *Constrained Clustering: Advances in Algorithms, Theory, and Applications*, 2008.
- [21] Nicolas Durand, Bruno Crémilleux, Ecclat: a new approach of clusters discovery in categorical data, in: *Twenty-Second Annual International Conference Knowledge Based Systems and Applied Artificial Intelligence (ES 2002)*, Springer, 2003, pp. 177–190.
- [22] Thi-Bich-Hanh Dao, Khanh-Chuong Duong, Christel Vrain, A declarative framework for constrained clustering, in: *Proceedings of the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases*, 2013, pp. 419–434.
- [23] Thi-Bich-Hanh Dao, Khanh-Chuong Duong, Christel Vrain, A filtering algorithm for constrained clustering with within-cluster sum of dissimilarities criterion, in: *Proceedings of the 25th International Conference on Tools with Artificial Intelligence*, 2013, pp. 1060–1067.
- [24] Thi-Bich-Hanh Dao, Khanh-Chuong Duong, Christel Vrain, Constrained minimum sum of squares clustering by constraint programming, in: *Principles and Practice of Constraint Programming - 21st International Conference, CP 2015, Cork, Ireland, August 31 - September 4, 2015, Proceedings*, 2015, pp. 557–573.
- [25] Thi-Bich-Hanh Dao, Khanh-Chuong Duong, Christel Vrain, Constrained clustering by constraint programming, *Artif. Intell.* 244 (2017) 70–94.
- [26] D. Defays, An efficient algorithm for a complete link method, *Comput. J.* 20 (4) (1977) 364–366.
- [27] Michel Delattre, Pierre Hansen, Bicriterion cluster analysis, *IEEE Trans. Pattern Anal. Mach. Intell.* Pami-2 (4) (1980) 277–291.
- [28] Cynthia Dwork, Moritz Hardt, Toniann Pitassi, Omer Reingold, Richard S. Zemel, Fairness through awareness, in: *Innovations in Theoretical Computer Science 2012*, 2012, pp. 214–226.
- [29] Thi-Bich-Hanh Dao, Chia-Tung Kuo, S.S. Ravi, Christel Vrain, Ian Davidson, Descriptive clustering: ILP and CP formulations with applications, in: Jérôme Lang (Ed.), *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI 2018, July 13-19, 2018, Stockholm, Sweden*, ijcai.org, 2018, pp. 1263–1269.
- [30] Martin Davis, George Logemann, Donald W. Loveland, A machine program for theorem-proving, *Commun. ACM* 5 (7) (1962) 394–397.
- [31] O. du Merle, P. Hansen, B. Jaumard, N. Mladenovic, An interior point algorithm for minimum sum-of-squares clustering, *SIAM J. Sci. Comput.* 21 (4) (1999) 1485–1505.
- [32] Ian Davidson, S.S. Ravi, Clustering with constraints: feasibility issues and the k-means algorithm, in: Hillol Kargupta, Jaideep Srivastava, Chandrika Kamath, Arnold Goodman (Eds.), *Proceedings of the 2005 SIAM International Conference on Data Mining, SDM 2005, Newport Beach, CA, USA, April 21-23, 2005*, SIAM, 2005, pp. 138–149.
- [33] Ian Davidson, S.S. Ravi, The complexity of non-hierarchical clustering with instance and cluster level constraints, *Data Min. Knowl. Discov.* 14 (1) (2007) 25–61.
- [34] I. Davidson, S. Ravi, L. Shamis, A SAT-based framework for efficient constrained clustering, in: *Proceedings of the SIAM International Conference on Data Mining*, 2010, pp. 94–105.
- [35] Thi-Bich-Hanh Dao, Christel Vrain, Khanh-Chuong Duong, Ian Davidson, A framework for actionable clustering using constraint programming, in: Gal A. Kaminka, Maria Fox, Paolo Bouquet, Eyke Hüllermeier, Virginia Dignum, Frank Dignum, Frank van Harmelen (Eds.), *ECAI 2016 - 22nd European Conference on Artificial Intelligence*, in: *Frontiers in Artificial Intelligence and Applications*, vol. 285, IOS Press, 2016, pp. 453–461.
- [36] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: Evangelos Simoudis, Jiawei Han, Usama M. Fayyad (Eds.), *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, Portland, Oregon, USA, AAAI Press, 1996, pp. 226–231.
- [37] Douglas H. Fisher, Knowledge acquisition via incremental conceptual clustering, *Mach. Learn.* 2 (2) (1987) 139–172.
- [38] F. Focacci, A. Lodi, M. Milano, Cost-based domain filtering, in: *Proceedings of the 5th International Conference on Principles and Practice of Constraint Programming*, 1999, pp. 189–203.
- [39] Germán González-Almagro, Daniel Peralta, Eli De Poorter, José-Ramón Cano, Salvador García, Semi-Supervised Constrained Clustering: An in-Depth Overview, *Ranked Taxonomy and Future Research Directions*, 2023.
- [40] M.A. Gluck, J.E. Corter, Information uncertainty and the utility of categories, in: *Proceedings of the Seventh Annual Conference of Cognitive Science Society*, 1985, pp. 283–287.
- [41] Tias Guns, Thi-Bich-Hanh Dao, Christel Vrain, Khanh-Chuong Duong, Repetitive branch-and-bound using constraint programming for constrained minimum sum-of-squares clustering, in: *ECAI 2016 - 22nd European Conference on Artificial Intelligence*, in: *Frontiers in Artificial Intelligence and Applications*, vol. 285, IOS Press, 2016, pp. 462–470.
- [42] Xifeng Guo, Long Gao, Xinwang Liu, Jianping Yin, Improved deep embedded clustering with local structure preservation, in: *IJCAI 2017*, 2017, pp. 1753–1759.
- [43] Tias Guns, Siegfried Nijssen, Luc De Raedt, k-pattern set mining under constraints, *IEEE Trans. Knowl. Data Eng.* 25 (2) (2013) 402–418.
- [44] Carla P. Gomes, Bart Selman, Henry A. Kautz, Boosting combinatorial search through randomization, in: Jack Mostow, Chuck Rich (Eds.), *Proceedings of the Fifteenth National Conference on Artificial Intelligence and Tenth Innovative Applications of Artificial Intelligence Conference, AAAI 98, IAAI 98, July 26-30, 1998, Madison, Wisconsin, USA*, AAAI Press / The MIT Press, 1998, pp. 431–437.

- [45] Mathieu Guilbert, Christel Vrain, Thi-Bich-Hanh Dao, Marcilio C.P. de Souto, Anchored constrained clustering ensemble, in: International Joint Conference on Neural Networks, IJCNN, 2022, Padua, Italy, July 18–23, 2022, IEEE, 2022, pp. 1–8.
- [46] Pierre Hansen, Michel Delattre, Complete-link cluster analysis by graph coloring, *J. Am. Stat. Assoc.* 73 (362) (1978) 397–403.
- [47] Pierre Hansen, Brigitte Jaumard, Cluster analysis and mathematical programming, *Math. Program.* 79 (1–3) (1997) 191–215.
- [48] Dino Ienco, Ruggero G. Pensa, Deep triplet-driven semi-supervised embedding clustering, in: DS, Springer, 2019, pp. 220–234.
- [49] Roberto J. Bayardo Jr., Robert Schrag, Using CSP look-back techniques to solve real-world SAT instances, in: Benjamin Kuipers, Bonnie L. Webber (Eds.), *Proceedings of the Fourteenth National Conference on Artificial Intelligence and Ninth Innovative Applications of Artificial Intelligence Conference, AAAI 97, IAAI 97*, July 27–31, 1997, Providence, Rhode Island, USA, AAAI Press / The MIT Press, 1997, pp. 203–208.
- [50] Mehdi Khiari, Patrice Boizumault, Bruno Crémilleux, Constraint programming for mining n-ary patterns, in: David Cohen (Ed.), *Principles and Practice of Constraint Programming - CP 2010 - 16th International Conference, CP 2010*, St. Andrews, Scotland, UK, September 6–10, 2010. *Proceedings*, in: *Lecture Notes in Computer Science*, vol. 6308, Springer, 2010, pp. 552–567.
- [51] Nimit Kumar, Krishna Kummamuru, Semisupervised clustering with metric learning using relative comparisons, *IEEE Trans. Knowl. Data Eng.* 20 (4) (2008) 496–503.
- [52] Chia-Tung Kuo, S.S. Ravi, Thi-Bich-Hanh Dao, Christel Vrain, Ian Davidson, A framework for minimal clustering modification via constraint programming, in: Satinder Singh, Shaul Markovitch (Eds.), *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, February 4–9, 2017, San Francisco, California, USA, AAAI Press, 2017, pp. 1389–1395.
- [53] Thomas Andrew Lampert, Thi-Bich-Hanh Dao, Baptiste Lafabregue, Nicolas Serrette, Germain Forestier, Bruno Crémilleux, Christel Vrain, Pierre Gançarski, Constrained distance based clustering for time-series: a comparative and experimental study, *Data Min. Knowl. Discov.* 32 (6) (2018) 1663–1707.
- [54] Stuart P. Lloyd, Least squares quantization in PCM, *IEEE Trans. Inf. Theory* 28 (2) (1982) 129–136.
- [55] J.B. MacQueen, Some methods for classification and analysis of multivariate observations, in: L.M. Le Cam, J. Neyman (Eds.), *Proc. of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, University of California Press, 1967, pp. 281–297.
- [56] Jean-Philippe Métivier, Patrice Boizumault, Bruno Crémilleux, Mehdi Khiari, Samir Loudni, Constrained clustering using SAT, in: Jaakko Hollmén, Frank Klawonn, Allan Tucker (Eds.), *Advances in Intelligent Data Analysis XI - 11th International Symposium, IDA 2012*, Helsinki, Finland, October 25–27, 2012. *Proceedings*, 2012, pp. 207–218.
- [57] Ryszard S. Michalski, A theory and methodology of inductive learning, *Artif. Intell.* 20 (2) (1983) 111–161.
- [58] Marianne Mueller, Stefan Kramer, Integer linear programming models for constrained clustering, in: Bernhard Pfahringer, Geoffrey Holmes, Achim G. Hoffmann (Eds.), *Discovery Science - 13th International Conference, DS 2010*, Canberra, Australia, October 6–8, 2010. *Proceedings*, in: *Lecture Notes in Computer Science*, vol. 6332, Springer, 2010, pp. 159–173.
- [59] Ryszard S. Michalski, Robert E. Stepp, Automated construction of classifications: conceptual clustering versus numerical taxonomy, *IEEE Trans. Pattern Anal. Mach. Intell.* 5 (4) (1983) 396–410.
- [60] M. Ng, A note on constrained k-means algorithms, *Pattern Recognit.* 33 (3) (2000) 515–519.
- [61] N.-V.-D. Nghiem, C. Vrain, T.-B.-H. Dao, Knowledge integration in deep clustering, in: *ECMLPKDD 2022*, 2022.
- [62] Nguyen-Viet-Dung Nghiem, Christel Vrain, Thi-Bich-Hanh Dao, Ian Davidson, Constrained clustering via post-processing, in: Annalisa Appice, Grigoris Tsoumakas, Yannis Manolopoulos, Stan Matwin (Eds.), *Discovery Science - 23rd International Conference, DS 2020*, Thessaloniki, Greece, October 19–21, 2020. *Proceedings*, in: *Lecture Notes in Computer Science*, vol. 12323, Springer, 2020, pp. 53–67.
- [63] A. Ouali, S. Loudni, Y. Lebbah, P. Boizumault, A. Zimmermann, L. Loukil, Efficiently finding conceptual clustering models with integer linear programming, in: *Proceedings of the International Joint Conference on Artificial Intelligence*, 2016, pp. 647–654.
- [64] Christos H. Papadimitriou, Kenneth Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*, Prentice-Hall, 1982.
- [65] Jean-Charles Régim, Arc consistency for global cardinality constraints with costs, in: *Proceedings of the 5th International Conference on Principles and Practice of Constraint Programming*, 1999, pp. 390–404.
- [66] Luc De Raedt, Tias Guns, Siegfried Nijssen, Constraint programming for itemset mining, in: Ying Li, Bing Liu, Sunita Sarawagi (Eds.), *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Las Vegas, Nevada, USA, August 24–27, 2008, ACM, 2008, pp. 204–212.
- [67] Yazhou Ren, Kangrong Hu, Xinyi Dai, Lili Pan, Steven C.H. Hoi, Zenglin Xu, Semi-supervised deep embedded clustering, *Neurocomputing* 325 (2019) 121–130.
- [68] Francesca Rossi, Peter van Beek, Toby Walsh (Eds.), *Handbook of Constraint Programming. Foundations of Artificial Intelligence*, Elsevier B.V., Amsterdam, Netherlands, August 2006.
- [69] Tian Sang, Paul Beame, Henry A. Kautz, Performing bayesian inference by weighted model counting, in: AAAI, vol. 5, 2005, pp. 475–481.
- [70] Robin Sibson, SLINK: an optimally efficient algorithm for the single-link cluster method, *Comput. J.* 16 (1) (1973) 30–34.
- [71] Florian Schroff, Dmitry Kalenichenko, James Philbin Facenet, A unified embedding for face recognition and clustering, *CoRR*, arXiv:1503.03832 [abs], 2015.
- [72] João P. Marques Silva, Karem A. Sakallah, GRASP: a search algorithm for propositional satisfiability, *IEEE Trans. Comput.* 48 (5) (1999) 506–521.
- [73] G.S. Tseitin, On the Complexity of Derivation in Propositional Calculus, Springer Berlin Heidelberg, Berlin, Heidelberg, 1983, pp. 466–483.
- [74] Takeaki Uno, Tatsuya Asai, Yuzo Uchida, Hiroki Arimura, LCM: an efficient algorithm for enumerating frequent closed item sets, in: Bart Goethals, Mohammed Javeed Zaki (Eds.), *FIMI '03, Frequent Itemset Mining Implementations*, *Proceedings of the ICDM 2003 Workshop on Frequent Itemset Mining Implementations*, 19 December 2003, Melbourne, Florida, USA, in: *CEUR Workshop Proceedings*, vol. 90, CEUR-WS.org, 2003.
- [75] Wouter Van Gansbeke, Simon Vandenhende, Stamatios Georgoulis, Marc Proesmans, Luc Van, Gool Scan, Learning to classify images without labels, in: *ECCV*, 2020, pp. 268–285.
- [76] K. Wagstaff, C. Cardie, Clustering with instance-level constraints, in: *Proceedings of the 17th International Conference on Machine Learning*, 2000, pp. 1103–1110.
- [77] K. Wagstaff, C. Cardie, S. Rogers, S. Schroedl, Constrained k-means clustering with background knowledge, in: *Proceedings of the International Conference on Machine Learning*, 2001, pp. 577–584.
- [78] Junyuan Xie, Ross B. Girshick, Ali Farhadi, Unsupervised deep embedding for clustering analysis, in: Maria-Florina Balcan, Kilian Q. Weinberger (Eds.), *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016*, New York City, NY, USA, June 19–24, 2016, in: *JMLR Workshop and Conference Proceedings*, vol. 48, JMLR.org, 2016, pp. 478–487.
- [79] Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, Guy Broeck, A semantic loss function for deep learning with symbolic knowledge, in: *ICML*, 2018, pp. 5502–5511.
- [80] Lintao Zhang, Sharad Malik, The quest for efficient boolean satisfiability solvers, in: Ed Brinksma, Kim Guldstrand Larsen (Eds.), *Computer Aided Verification, 14th International Conference, CAV 2002*, Copenhagen, Denmark, July 27–31, 2002. *Proceedings*, in: *Lecture Notes in Computer Science*, vol. 2404, Springer, 2002, pp. 17–36.
- [81] Hongjing Zhang, Tianyang Zhan, Sugato Basu, Ian Davidson, A framework for deep constrained clustering, *Data Min. Knowl. Discov.* 35 (2) (2021) 593–620.